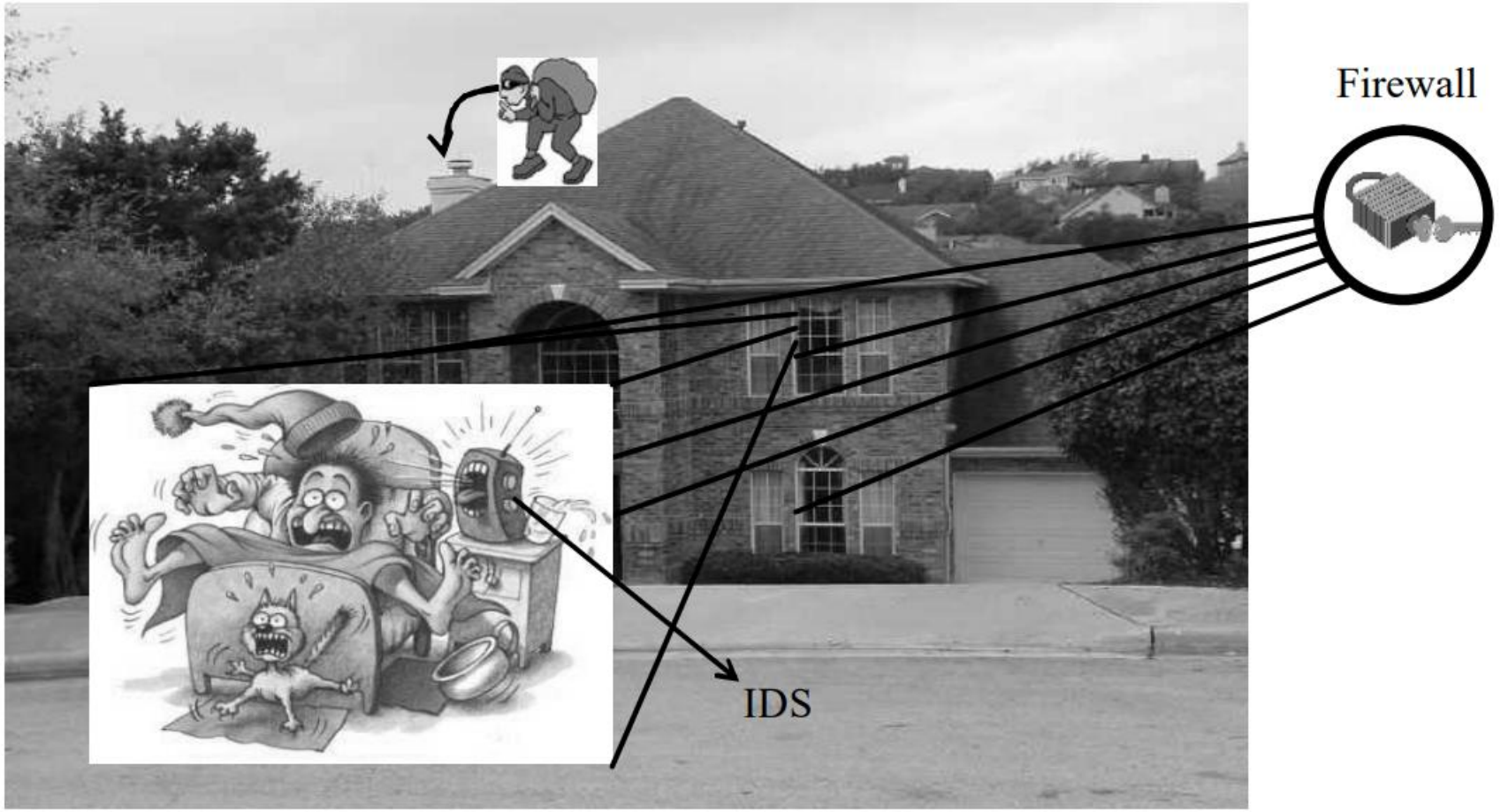
بسمه تعالی

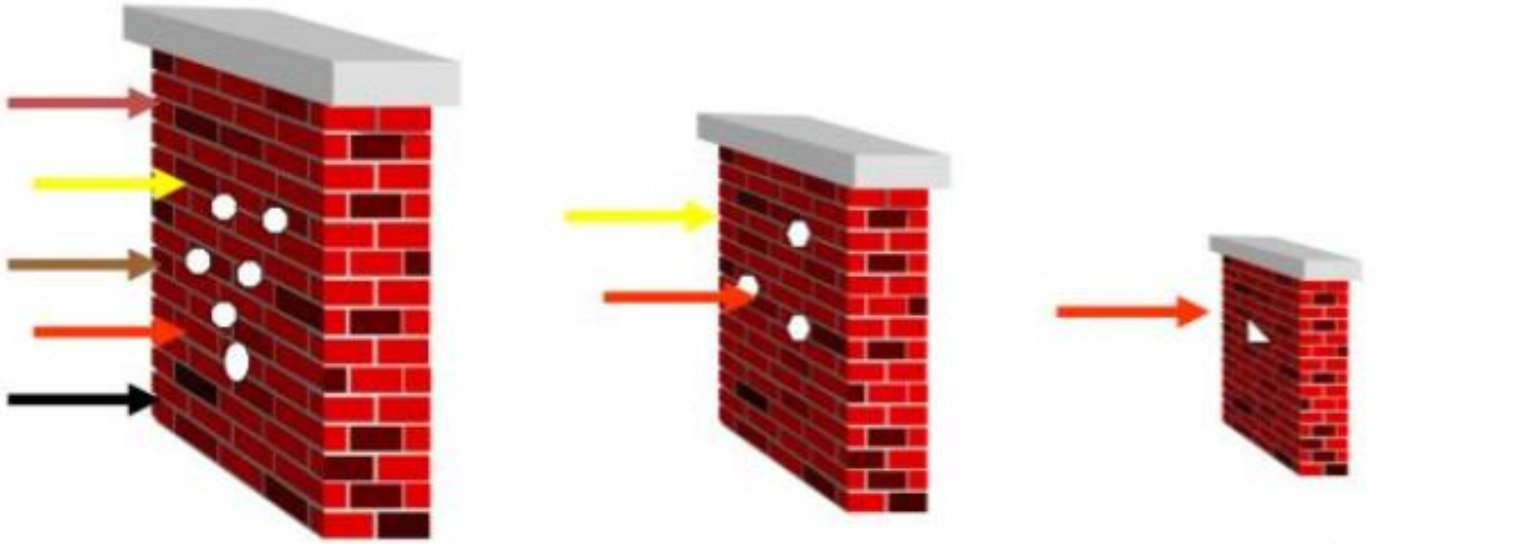


تشخیص نفوذ
درس دوم : ساختار XDR

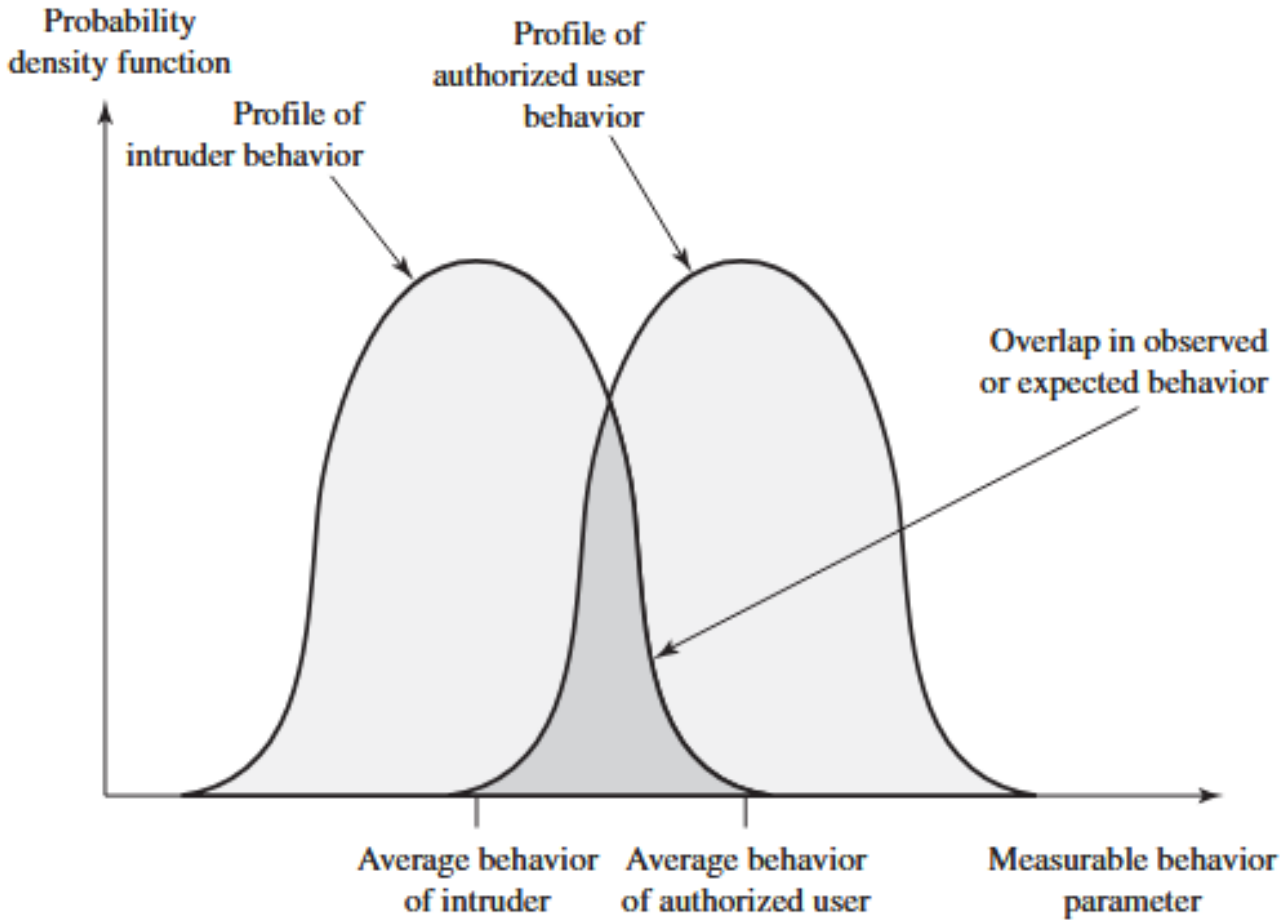
چرا دیوار آتش کافی نیست؟



چرا دیوار آتش کافی نیست؟

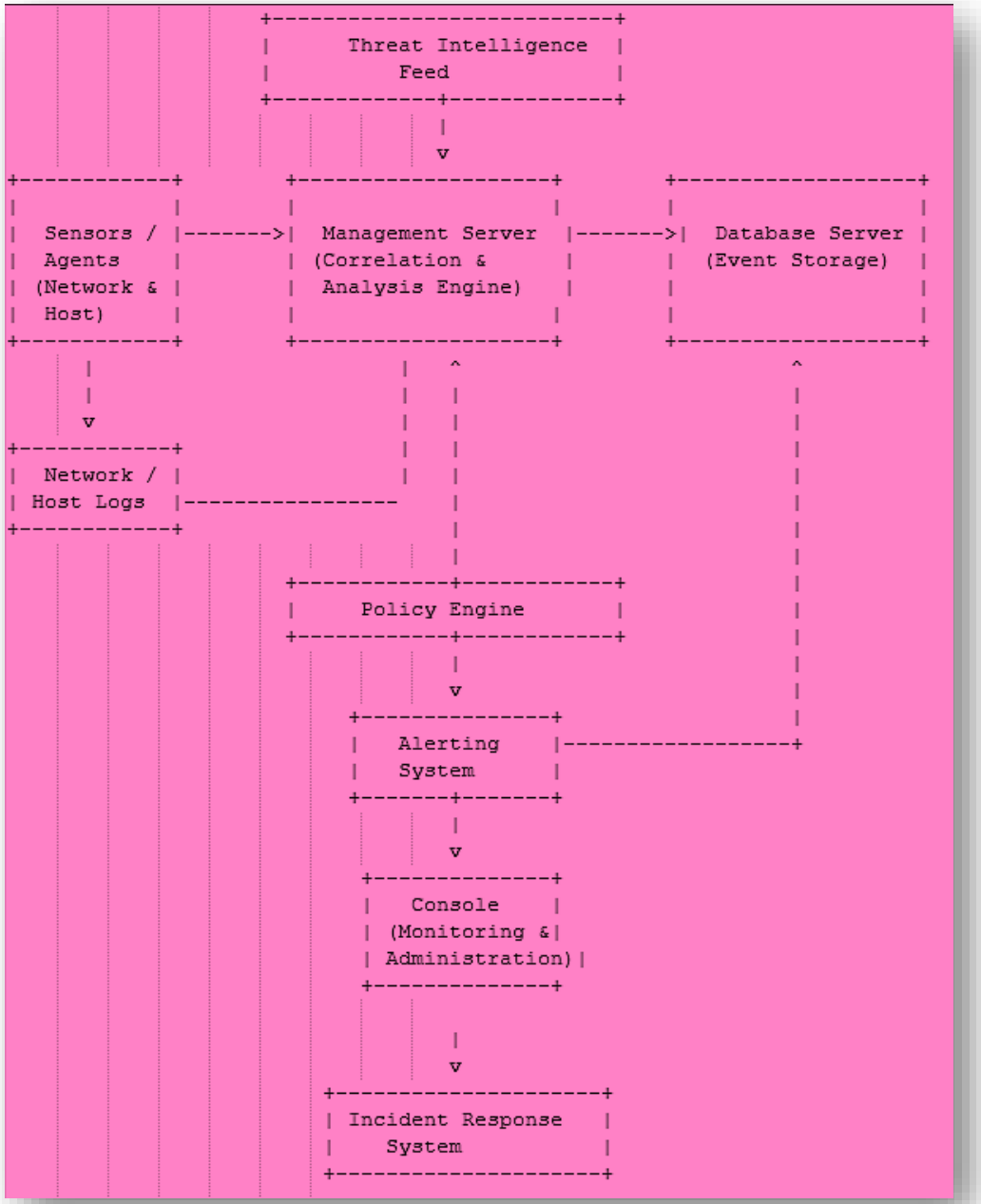


دو فرض اصلی سیستم های تشخیص نفوذ



- فرض مشاهده پذیر بودن فعالیت ها
  - در شبکه کشف منابع چالش است
- فرض قابل جداسازی بودن نفوذ از رفتار عادی
  - در سیستم یافتن نقطه طلایی مشاهده چالش است.

- در این بخش از درس این دو مساله را مفروض میگیریم گرچه در درس های بعدی درباب تحقق و امکان پذیری آنها بحث می شود.



اجزای سیستم تشخیص نفوذ

- حسگرها/عامل‌ها داده‌ها را از ترافیک شبکه یا لاگ‌های میزبان جمع‌آوری می‌کنند.
  - اگر حسگرهای سطح میزبان همه چیز را در سطح یک میزبان می بینند، چرابه حس گر شبکه نیازمندیم؟
  - آنها داده‌های خام رویداد را به سرور مدیریت ارسال می‌کنند.
- سرور مدیریت، همبستگی و تجزیه و تحلیل را انجام می‌دهد، گاهی اوقات با استفاده از اطلاعات از فید هوش تهدید و با هدایت قوانین موتور سیاست
- داده‌های رویداد برای تجزیه و تحلیل تاریخی در سرور پایگاه داده ذخیره می‌شوند.
- سیستم هشدار، اعلان‌هایی را در مورد فعالیت‌های مشکوک ارسال می‌کند.
- کنسول رابطی را برای مدیران فراهم می‌کند تا نظارت، پیکربندی و پاسخ دهند.
- سیستم پاسخ به حادثه، گردش‌های کاری پاسخ خودکار/دستی را مدیریت می‌کند.

به نظر شما وجود همه این اجزا در یک ماشین منطقی است؟

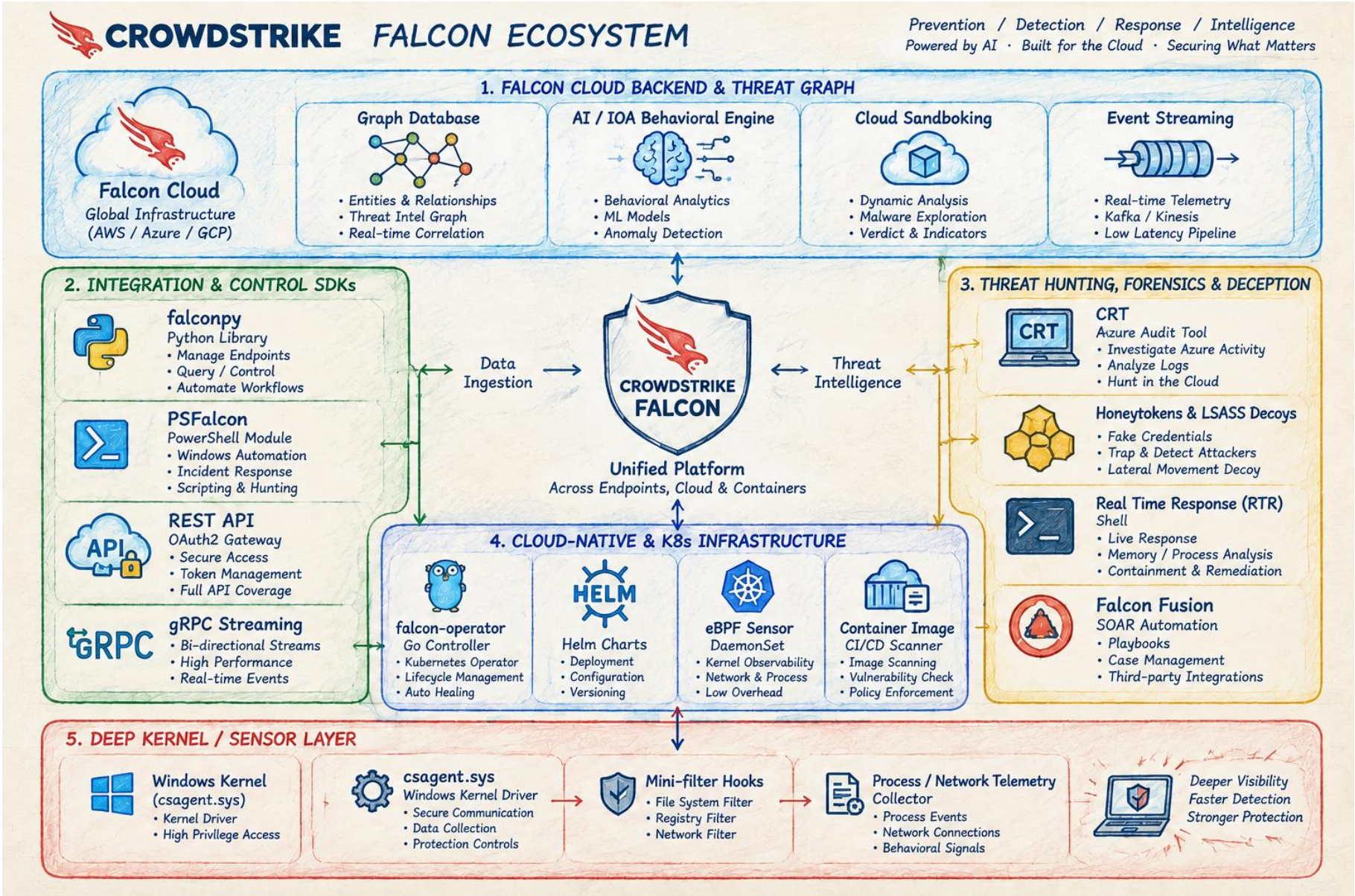


تفاوت یک سیستم تشخیص نفوذ چند صددلاری و چندهزار دلاری در چیست ؟

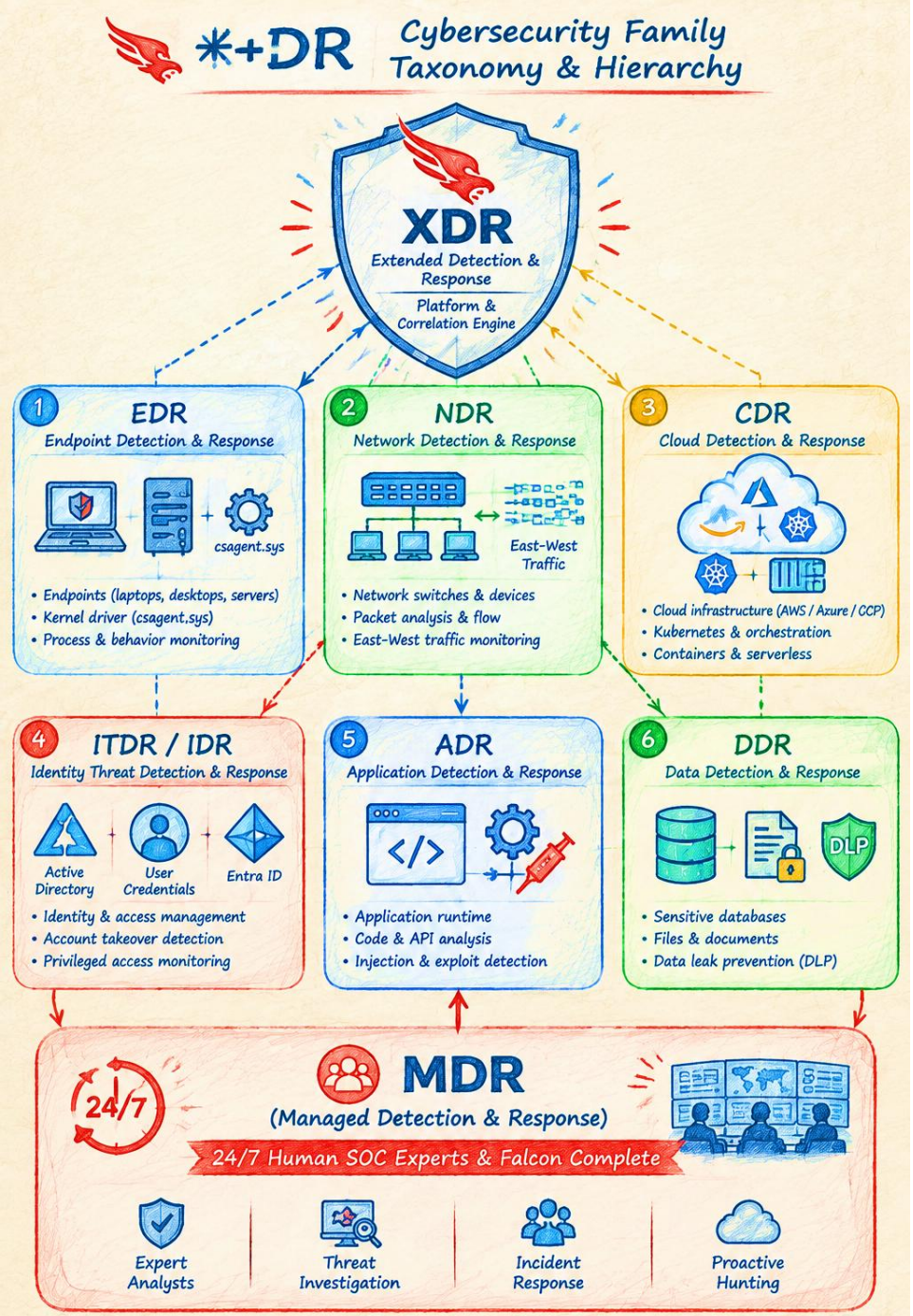


- اشتراک Businessقوانین Snortبه‌ازای هر سنسور فروخته می‌شود و یک اشتراک ۳۹۹ دلاری می‌تواند کل شبکه با هر تعداد دستگاه را پوشش دهد، در حالی که فالکون به‌ازای هر دستگاه و به‌صورت سالانه لایسنس می‌خواهد. در نتیجه برای سازمانی با ۵۰۰ دستگاه، نسخه اصلی EDR فالکون حدود ۹۲٬۵۰۰ دلار در سال هزینه دارد؛ یعنی تقریباً ۲۳۰ برابر اشتراک اسنورت.
- این اختلاف بازتاب تفاوت عمیق در معماری و دامنه پوشش دو محصول است، نه صرفاً سیاست قیمت‌گذاری متفاوت.

اجزای فالکون EDR



- در مستندات فالکون هیچ جا چنین تصویر کاملی شرح داده نشده، به نظر شما از کجا می توان به چنین دیاگرام دقیقی رسید؟



- **۱. Endpoint Detection and Response —EDR** (پایه و هسته اصلی)
- **قلمرو:** لپ‌تاپ‌ها، سرورها، ایستگاه‌های کاری).
- **کارکرد:** ثبت تمامی رفتارهای سطح هسته (Kernel) مانند ایجاد پروسه‌ها، تغییرات رجیستری، باز شدن سوکت شبکه و دستکاری فایل‌ها؛ تحلیل رفتاری (IOA) و پاسخ بلادرنگ (Kill پروسه یا ایزوله کردن کلاینت).
- **۲. Network Detection and Response —NDR**
- **قلمرو:** ترافیک خام شبکه، تحلیل جریان‌ها و بسته‌ها.
- **کارکرد:** رصد ارتباطات جانبی مهاجم، ارتباط با سرورهای فرماندهی و کنترل (۲C) و نشت داده، بدون نیاز به داشتن ایجنت روی تک‌تک دیوایس‌ها (مثل سوییچ‌ها، پرینترها و تجهیزات بدون سنسور).
- **۳. Cloud Detection and Response —CDR**
- **قلمرو:** محیط‌های ابری، زیرساخت‌های AWS, Azure, GCP، کانتینرها و کلاسترهای Kubernetes.
- **کارکرد:** کشف رفتارهای غیرعادی در کنترل پلین ابری، دستکاری منابع، فرار از کانتینر و ناهنجاری‌های زیرساخت ابری.
- **۴. Identity Threat Detection and Response —IDR / ITDR**
- **قلمرو:** دایرکتوری‌ها، کنترل‌کننده‌های دامنه و احراز هویت.
- **کارکرد:** تشخیص سرقت اعتبارنامه‌ها، حملات Pass-the-Hash، Kerberoasting، ترفیع غیرمجاز دسترسی و رفتارهای غیرعادی لاگین کاربران.
- **۵. Application Detection and Response —ADR**
- **قلمرو:** برنامه‌های کاربردی در زمان اجرا (Runtime Application Self-Protection / RASP).
- **کارکرد:** رصد حملات در لایه لاجیک اپلیکیشن، مانند تزریق کد (SQLi, XSS)، سوءاستفاده از کتابخانه‌های آسیب‌پذیر (مثل بحران j۴Log) و دستکاری حافظه پروسه وب‌سرور.
- **۶. Data Detection and Response —DDR**
- **قلمرو:** داده‌های حساس سازمان (اسناد، پایگاه‌های داده، سورس‌کدها، فایل‌های ابری).
- **کارکرد:** ردگیری جابه‌جایی و سرقت داده‌های محرمانه (Exfiltration)، تشخیص فازهای اولیه باج‌افزار قبل از رمزگذاری انبوه، و حفاظت از نشت داده (DLP نسل جدید).

نیازمندی های یک سیستم تشخیص نفوذ

- پیوسته اجرا شود
- مقاوم در برابر خطا
- مقاومت در برابر براندازی
- تحمیل حداقل سربار به سیستم
- پیکربندی براساس سیاست های امنیتی سیستم
- سازگاری با تغییرات سیستم ها و کاربران
- مقیاس پذیر برای نظارت بر تعداد زیادی از سیستم ها
- ارائه خدمات مطلوب
- امکان پیکربندی مجدد پویا

نیازمندی های یک سیستم تشخیص نفوذ

- پیوسته اجرا شود
- مقاوم در برابر خطا
- مقاومت در برابر براندازی
- تحمیل حداقل سربار به سیستم
- پیکربندی براساس سیاست های امنیتی سیستم
- سازگاری با تغییرات سیستم ها و کاربران
- مقیاس پذیر برای نظارت بر تعداد زیادی از سیستم ها
- ارائه خدمات مطلوب

آیا می توانید مثال هایی برای نقض این موارد و ریسک ناشی از آنها بزنید؟

نیازمندی های یک سیستم تشخیص نفوذ

سیستم تشخیص نفوذی که پس از هر به‌روزرسانی نیاز به راه‌اندازی مجدد دارد، باعث ایجاد شکاف‌های نظارتی در طول راه‌اندازی مجدد می‌شود. مهاجمان از این پنجره‌ها برای نفوذ بدون شناسایی سوءاستفاده می‌کنند.

- پیوسته اجرا شود

اجرای پیوسته

Hot Reloading of Rules/Signatures

باید به شکلی برنامه بدون ریستارت شدن فایل پیکربندی خود رادوباره بخواند
مثلا درلینوکس با ارسال سگینال SIGHUB به برنامه امکانپذیر است

Stateful Rule Engine with Atomic Swapping

دو مجموعه قانون را در حافظه نگه داریم: فعلی و در انتظار.
دربه‌روزرسانی قوانین جدید را در مجموعه در انتظار تجزیه و اعتبارسنجی کنید.پس از اعتبارسنجی، اشاره‌گرها را به صورت اتمی مبادله کنید تا در انتظار به فعلی تبدیل شوند.مهم:از) RCUخواندن-کپی-به‌روزرسانی) یا ساختارهای داده بدون قفل برای جلوگیری از مسدود شدن پردازش بسته استفاده می‌کند.

Dynamic Plugin/Module Loading (For HIDS & Advanced NIDS)

موتور اصلی همچنان در حال اجرا است؛ منطق تشخیص به صورت کتابخانه‌های مشترک بارگذاری می‌شود

Multi-Process or Multi-Threaded Architecture with Graceful Handoff

دو نمونه از موتور IDS اجرا می شود:

- فعال: پردازش ترافیک زنده.
- آماده به کار: بارگذاری قوانین/پیکربندی جدید در پس‌زمینه.

استفاده از یک سوئیچ ترافیک (مثلاً iptables failover، eBPFیا load balancer) برای انتقال فوری ترافیک به نمونه به‌روزرسانی‌شده.

Containerized / Microservices Deployment (Cloud-Native)

service meshجدید با قوانین به‌روزرسانی‌شده شروع می‌شود.ترافیک از طریق Podها به‌روزرسانی کنید:Podبا راه‌اندازی مجدد
قدیمی به طرز ماهرانه‌ای اتصالات را تخلیه می‌کند. Podتغییر می‌کند. kube-proxyیا (Istioمثلاً)

نیازمندی های یک سیستم تشخیص نفوذ

- پیوسته اجرا شود
- مقاوم در برابر خطا

یک حسگر به دلیل نشت حافظه از کار می‌افتد اما به طور خودکار راه‌اندازی مجدد نمی‌شود. در نتیجه، بخش شبکه‌ای که آن را رصد می‌کند، ساعت‌ها بدون محافظت رها می‌شود.

Chaos engineering و تزریق خطا
فازینگ و تست قوی
...

نیازمندی های یک سیستم تشخیص نفوذ

یک مهاجم دسترسی ادمین را به دست می‌آورد و عامل میزبان را غیرفعال می‌کند یا پیکربندی آن را تغییر می‌دهد و عملاً سیستم را نسبت به فعالیت خود کور می‌کند.

- پیوسته اجرا شود
- مقاوم در برابر خطا
- مقاوم در برابر براندازی

Self-protection
- **Self-Monitoring & Self-Healing**
- **Immutable/Versioned Rule Sets + Rule Signatures (both)**
- **Tamper-Evident Local Logs: Append-Only + Merkle Trees (both)**
- **Redundancy**
- **Kernel driver**
- **Runtime Integrity Checks & Anti-Debugging (both)**
- **Secure update**

Boot/OS protection
- **Process protection**
- **Watchdog monitoring**

Anti-evasion
- **Handle encryption**
- **State fullness**
- **…**

نیازمندی های یک سیستم تشخیص نفوذ

- پیوسته اجرا شود
- مقاوم در برابر خطا
- مقاوم در برابر براندازی
- تحمیل حداقل سربار به سیستم

با استفاده سنگین از IDPS یک و حافظه، سرورهای حیاتی را CPU کند می‌کند و باعث می‌شود مدیران عملکرد آن را غیرفعال یا محدود کنند و در نتیجه، دید امنیتی را کاهش دهند.



- شبکه قطع شده ؟
- نه
- به بانک حمله شده ؟
- نه
- سیستم تشخیص نفوذ جدید نصب شده !

نیازمندی های یک سیستم تشخیص نفوذ

- پیوسته اجرا شود
- مقاوم در برابر خطا
- مقاوم در برابر براندازی
- تحمیل حداقل سربار به سیستم
- پیکربندی بر اساس سیاست های امنیتی

قوانین تشخیص پیش‌فرض و عمومی باعث سیلی از تشخیص‌های مثبت کاذب می‌شوند که منجر به خستگی از هشدارها می‌شود. در نهایت، هشدارهای واقعی نادیده گرفته می‌شوند یا از دست می‌روند.

Tuning the context

یک IDSتنظیم‌شده با ۲۰ قانون، مؤثرتر از یک IDSنویزدار با ۲۰۰۰۰ قانون است.

پس قبل از به کارگیری سیستم تشخیص نفوذ باید سیستم به خوبی فهمیده شود

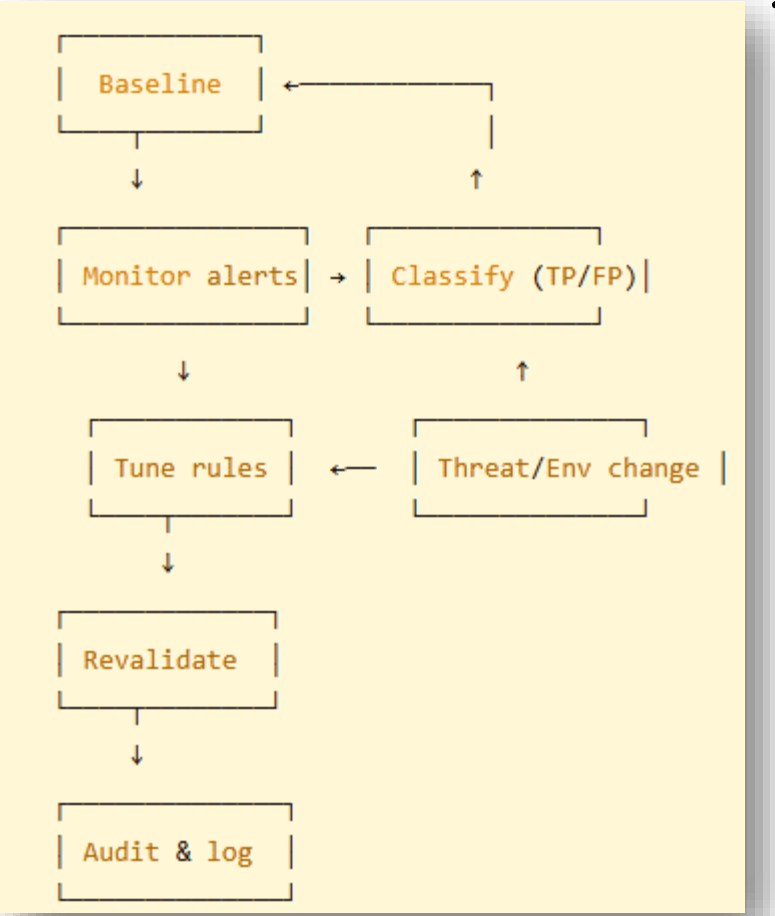
حالت های عادی غیرممکن استخراج شوند

این حالت ها در یک کمینه قوانین گنجانده شوند

از همان ابتدا قوانینی که باید از پیشفرض خارج باشندsupress شوند

tuning :
suppress,
threshold,
update rules

نیازمندی های یک سیستم تشخیص نفوذ



- پیوسته اجرا شود
- مقاوم در برابر خطا
- مقاوم در برابر براندازی
- تحمیل حداقل سربار به سیستم
- پیکربندی بر اساس سیاست های امنیتی
- سازگاری با تغییرات سیستم ها و کاربران

ماشین‌های مجازی یا نمونه‌های ابری جدید راه‌اندازی می‌شوند اما به طور خودکار به مانیتورینگ اضافه نمی‌شوند و همین امر باعث ایجاد نقاط کوری می‌شود که توسط مهاجمان مورد سوءاستفاده قرار می‌گیرند.

نیازمندی های یک سیستم تشخیص نفوذ

- پیوسته اجرا شود
- مقاوم در برابر خطا
- مقاوم در برابر براندازی
- تحمیل حداقل سربار به سیستم
- پیکربندی بر اساس سیاست های امنیتی
- سازگاری با تغییرات سیستم ها و کاربران
- مقیاس پذیر برای نظارت بر تعداد زیادی از سیستم ها

رشد ناگهانی شبکه، سرور مدیریت را با بار اضافی مواجه می‌کند و باعث از دست رفتن رویدادها و تأخیر در هشدارها می‌شود و به حملات اجازه می‌دهد تا بدون جلب توجه ادامه یابند.

نیازمندی های یک سیستم تشخیص نفوذ

- پیوسته اجرا شود
- مقاوم در برابر خطا
- مقاوم در برابر براندازی
- تحمیل حداقل سربار به سیستم
- پیکربندی بر اساس سیاست های امنیتی
- سازگاری با تغییرات سیستم ها و کاربران
- مقیاس پذیر برای نظارت بر تعداد زیادی از
- ارائه خدمات مطلوب

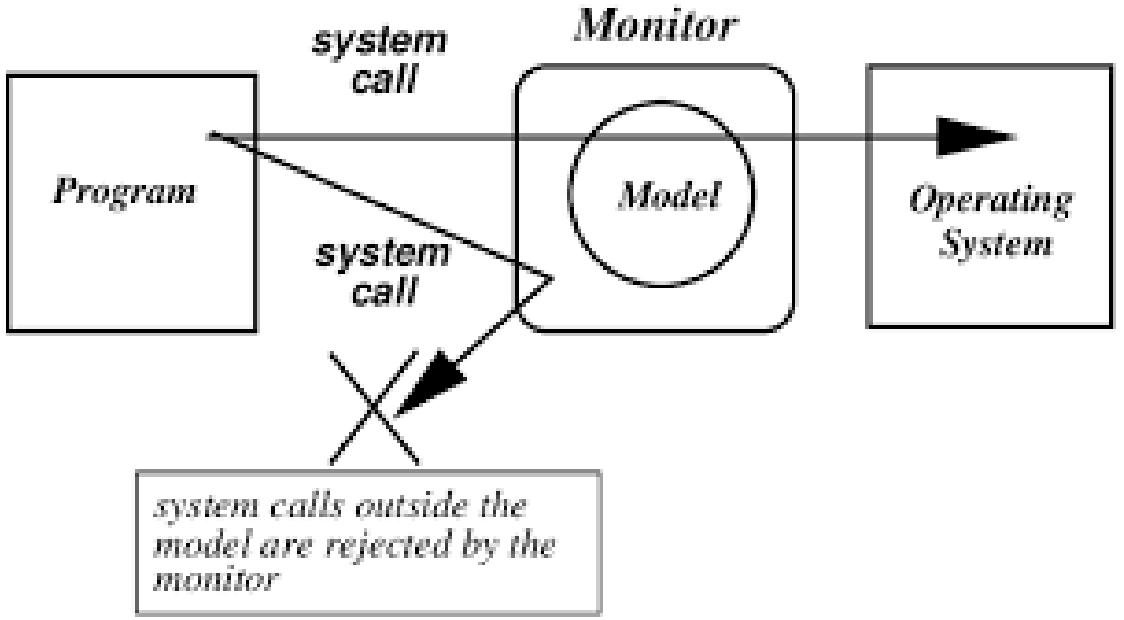
ادامه درس!

عملکرد سیستم تشخیص نفوذ به چه شکل است؟

بخش تحلیلگر سیستم های تشخیص نفوذ اغلب یک برنامه نرم افزاری یا سخت افزاری است که

- ارضا شدن یک قانون
- تطبیق با یک سیاست امنیتی
- تطبیق با یک امضا
- تطبیق با یک نشانه از یک رفتار

را بررسی می کند و در صورت برقرار بودن آن هشدار می دهد (یا در مواردی که به صورت پیشگیرانه نصب شده باشد، می تواند عکس العمل نشان دهد.

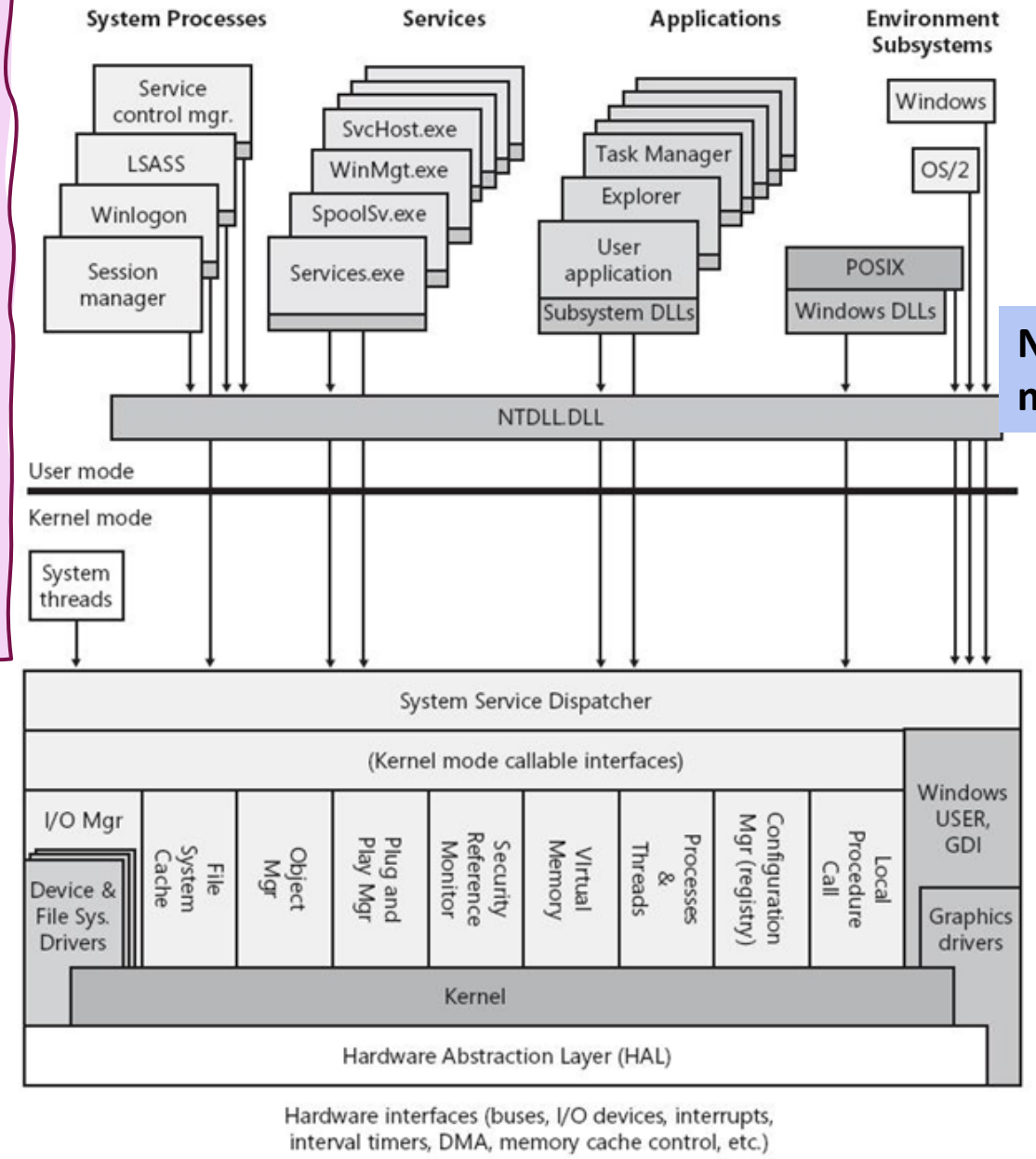


رصد سیستم و فراخوانی های سیستمی

مولفه ای وابسته به سیستم

معماری سیستم عامل

- **high-level, user-friendly API**
- stable, documented interface
- offers **extra features** like Unicode handling, heap management, synchronization objects, and higher-level file functions that combine multiple Native API calls.
- can add **validation, error checking, and workarounds** before calling the Native API.



Native API more volatile;

Reprinted, by permission, from *Inside Microsoft Windows 2000, 3rd Edition* (ISBN 0-7356-1021-5). © 2000 by David A Solomon and Mark E. Russinovich. All rights reserved.

فراخوانی سیستمی

[VISITOR LEVEL]

App (Your Program)

"Please open this file."

user32.dll / kernel32.dll
Front Desk Staff
- Validates request
- Packages it into company language
- Decides which department to call

ntdll.dll
Secure Messenger
- Puts request in sealed envelope
- Speaks low-level "company protocol"
- Walks to secure elevator



↓↓↓ SECURE ELEVATOR ↓↓↓

[syscall — The Secure Transport]

[EXECUTIVE FLOOR]

Dispatcher (KiSystemService / SSDT)
Elevator Operator
- Receives messenger
- Routes to exact office/desk

ntoskrnl.exe (NT Kernel)
Executive Office / Operations Floor
- Makes core decisions
- Schedules processes, allocates memory, enforces security policies

Drivers (Network, Disk, GPU, etc.)
Specialist Technicians
- Execute orders from executives
- Operate physical machines

HAL (Hardware Abstraction Layer)
Translator
- Converts executive commands into device-specific instructions (e.g., "write sector" → SATA/IDE/NVMe)

[PHYSICAL LAYER]

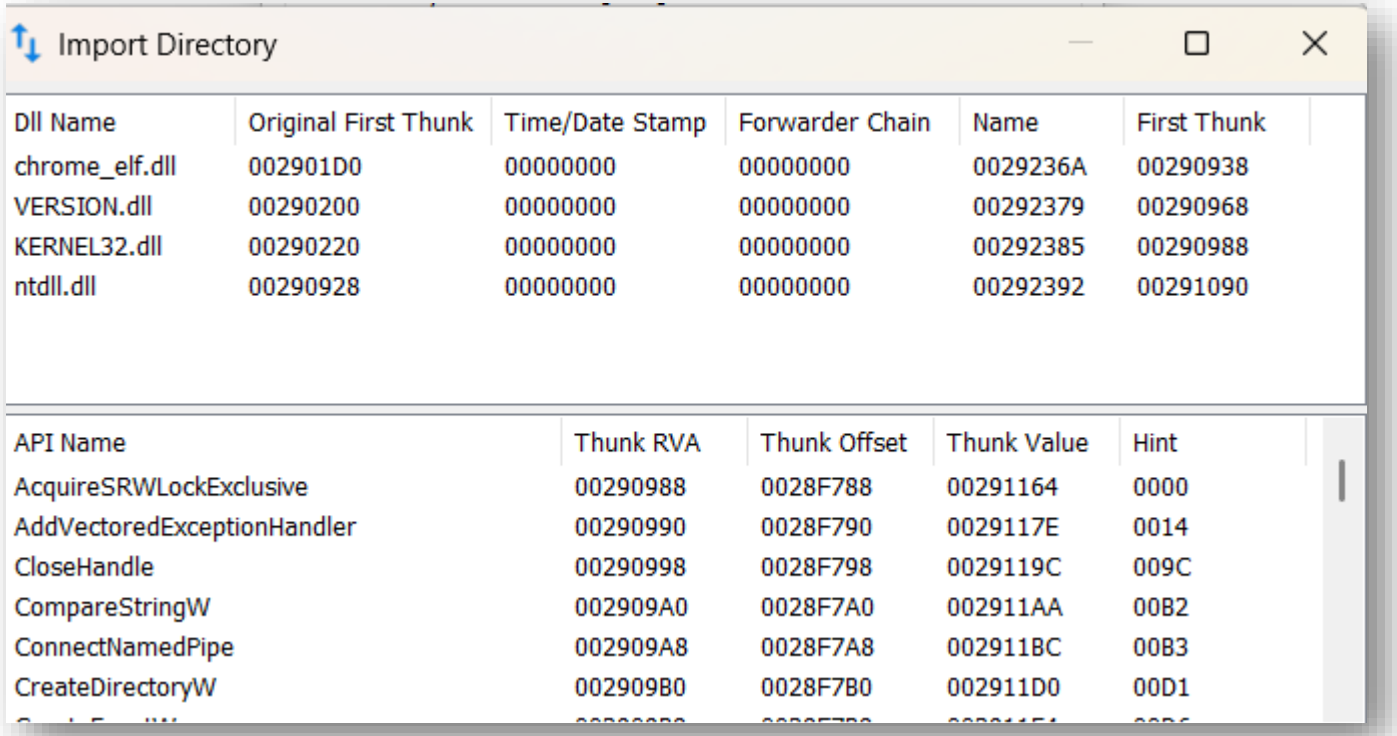
Hardware (CPU, RAM, Disk, NIC, GPU, Sensors)

Physical Machines
- Execute actual I/O, compute, store data
- Speak only in voltages and signals

آیا راهی برای مشاهده فراخوانی های سیستمی یک برنامه (برای سیستم عامل یا HIDS) قبل از اجرای برنامه هست؟

- بدون disassembleکردن برنامه ؟
  چه ایستا و چه پویا ؟

ساده ترین روش نظارت بر فراخوانی های سیستمی
خواندن IATهای برنامه: ایستا و بدون نیاز به اجرا



رفتار یک برنامه نرمال کامپایل شده

- وقتی برنامه شما تابعی را از یک DLLفراخوانی می‌کند (مثلاً MessageBoxAاز user32.dll)، آدرس دقیق حافظه آن تابع را نمی‌داند.
- IATبه عنوان یک جدول جستجو عمل می‌کند: در طول بارگذاری برنامه، ویندوز IATرا با آدرس‌های واقعی توابع وارد شده پر می‌کند.
- سپس کد شما تابع را به طور غیرمستقیم از طریق IAT فراخوانی می‌کند.
- مزایا:
- ماژولار بودن: می‌توانید DLLها را بدون کامپایل مجدد برنامه به‌روزرسانی کنید.
- استفاده مجدد از کد: چندین برنامه می‌توانند DLL یکسانی را در حافظه به اشتراک بگذارند.
- امنیت و وصله‌گذاری: به‌روزرسانی‌های سیستم عامل می‌توانند DLLها را به صورت مرکزی وصله‌گذاری کنند.

ساده ترین روش نظارت بر فراخوانی های سیستمی
خواندن IATهای برنامه: ایستا و بدون نیاز به اجرا

- وقتی برنامه شما تابعی را از یک DLLفراخوانی می‌کند (مثلاً MessageBoxAاز user32.dll)، آدرس دقیق حافظه آن تابع را نمی‌داند.
- IATبه عنوان یک جدول جستجو عمل می‌کند: در طول بارگذاری برنامه، ویندوز IATرا با آدرس‌های واقعی توابع وارد شده پر می‌کند.
- سپس کد شما تابع را به طور غیرمستقیم از طریق IAT فراخوانی می‌کند.
- مزایا:
- ماژولار بودن: می‌توانید DLLها را بدون کامپایل مجدد برنامه به‌روزرسانی کنید.
- استفاده مجدد از کد: چندین برنامه می‌توانند DLL یکسانی را در حافظه به اشتراک بگذارند.
- امنیت و وصله‌گذاری: به‌روزرسانی‌های سیستم عامل می‌توانند DLLها را به صورت مرکزی وصله‌گذاری کنند.

Import Directory

Dll Name	Original First Thunk	Time/Date Stamp	Forwarder Chain	Name	First Thunk
chrome_elf.dll	002901D0	00000000	00000000	0029236A	00290938
VERSION.dll	00290200	00000000	00000000	00292379	00290968
KERNEL32.dll	00290220	00000000	00000000	00292385	00290988
ntdll.dll	00290928	00000000	00000000	00292392	00291090

API Name	Thunk RVA	Thunk Offset	Thunk Value	Hint
AcquireSRWLockExclusive	00290988	0028F788	00291164	0000
AddVectoredExce	0990	0028F790	0029117E	0014
CloseHandl		0028F798	0029119C	009C
Compa		0028F7A0	002911AA	00B2
Con		28F7A8	002911BC	00B3
Cre		8F7B0	002911D0	00D1

آیا در این جدول ترتیب فراخوانی ها و آرگمان ها مشخص است؟
آیا امکان نظارت فعال است یا منفعل؟

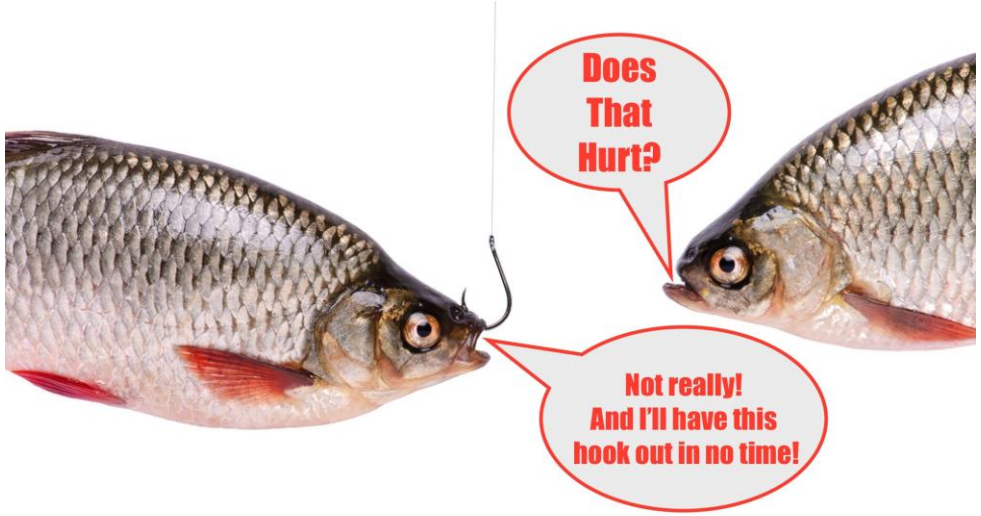
سیستم عامل میتواند دسترسی مداخله در فراخوانی سیستمی به برنامه بدهد

در ویندوز ETW (Event Tracing)

سیستم عامل میتواند دسترسی مداخله در فراخوانی سیستمی به برنامه بدهد

- در ویندوز ETW (Event Tracing)
- ویندوز سال‌هاست ابزارهایی مثل **ETW (Event Tracing for Windows)** دارد، اما ETW فقط **خواندنی** (**Read-Only**) و پس از واقعه است (شما فقط می‌فهمید چه شد، نمی‌توانید جلوی آن را همان لحظه داخل کرنل بگیرید). برای مسدودسازی آنی در ویندوز،

- نیازمند اعتماد به سیستم عامل
- حمله کننده ممکن است پیکربندی را تغییر داده و این رهگیری ها را دور بزند
- نیازمند دسترسی ادمین



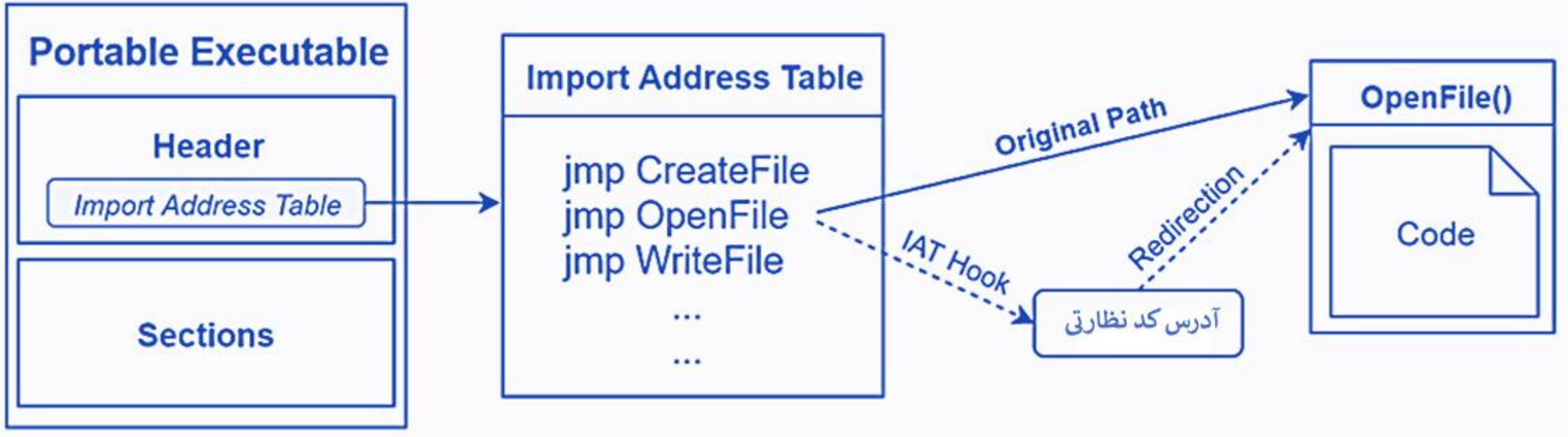
خود برنامه تشخیص نفوذ می تواند یک مولفه نظارتی زمان اجرا برای نظارت بر فراخوانی های سیستمی داشته باشد

- این کار اغلب با تکنیک های قلاب سازی (hook) ممکن است.
- هوکینگ (Hooking) تکنیکی است که برای رهگیری یا تغییر رفتار فراخوانی‌های تابع، پیام‌ها یا رویدادها در یک سیستم نرم‌افزاری استفاده می‌شود.
- این تکنیک به یک توسعه‌دهنده (یا مهاجم) اجازه می‌دهد تا به یک فرآیند "هوک" کند و **کد دلخواه را قبل، بعد یا به جای تابع اصلی** اجرا کند.
- نوعی حمله مرد میانی در سطح سیستم

نظارت فعال بر فراخوانی های سیستمی

هیچ چیز بجز خود برنامه تغییری نمی کند!

- قلاب کردن جدول IAT
- **قلاب کردن (hooking)** سطح کاربری (و نه اصل فراخوانی سیستمی).



میشه دورش زد؟

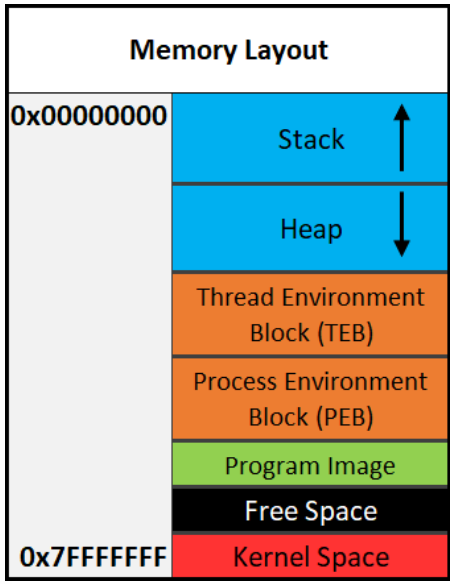
موش و گربه قلاب کردن و دور زدن قلاب

روش اول فراخوانی مستقیم توابع در جایی عمیق تر از سیستم

شرکت ما همه فراخوانی HIDS های تک تک برنامه ها را در حافظه چک میکند
(با قلاب سازی جدول IAT)

برای دسترسی به توابع kernel32 راه های متعددی هست. بدون استفاده از IAT آن فراخوانی حساس را صدا می زنم و محصول شما را دور می زنم

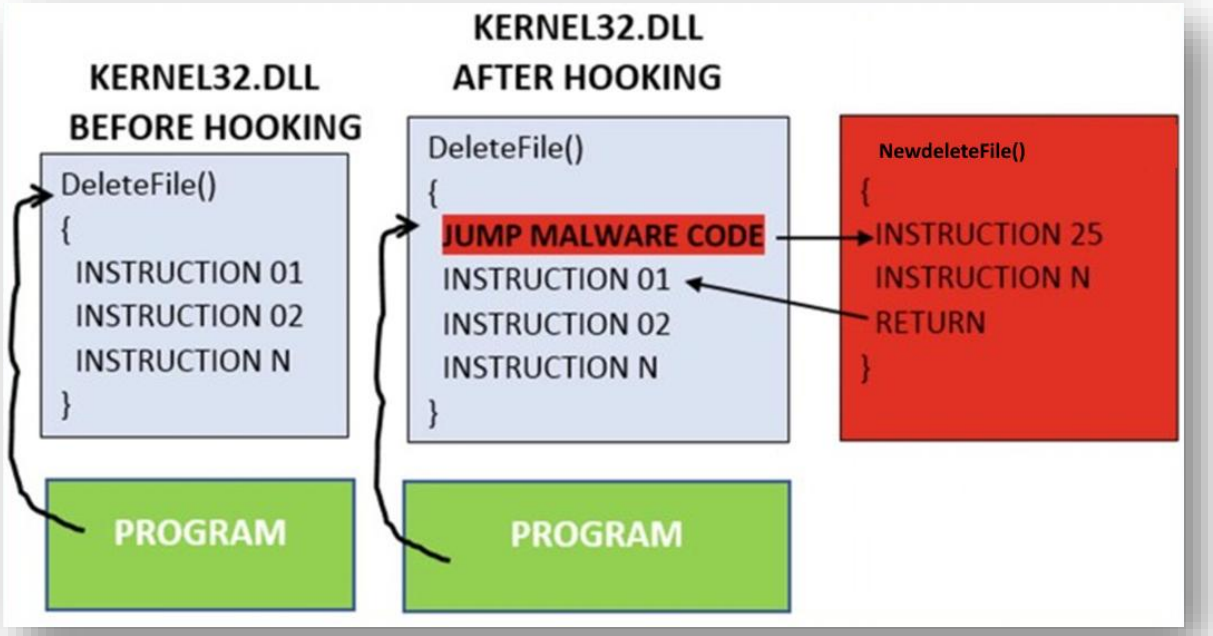
Technique	Works with ASLR?	Why / How
1. LoadLibrary + GetProcAddress	✅ Yes	These APIs return the correct base address and function pointer **after ASLR relocation**, so attackers can still resolve and call functions dynamically.
2. Hardcoded Function Pointers	❌ No (unless bypassed)	Hardcoding absolute addresses fails under ASLR because module base addresses are randomized. Attackers must first **resolve the base address dynamically** (e.g., via PEB) before computing offsets.
3. PEB Traversal	✅ Yes	The PEB contains the **actual base addresses** of loaded modules (post-ASLR). Malware can walk the PEB to find `kernel32.dll`, then parse its export table to resolve function addresses.
4. Shellcode / Reflective Loading	✅ Yes	Shellcode typically includes logic to resolve module base addresses and function pointers at runtime, making it **ASLR-resilient**. Reflective DLLs do the same internally.



HIDS شرکت ما همه فراخوانی های تک تک برنامه ها را در حافظه چک میکند
(با قلاب سازی جدول IAT)

برای دسترسی به توابع kernel32 راه های متعددی هست. بدون استفاده از IAT آن فراخوانی حساس را صدا می زنم و محصول شما را دور می زنم

شرکت ما تصمیم گرفته است با قلاب کردن kernel32هرآنچه در سیستم فراخوانی می شود را چک کند.



HIDS شرکت ما همه فراخوانی های تک تک برنامه ها را در حافظه چک میکند
(با قلاب سازی جدول IAT)

شرکت ما تصمیم گرفته است با قلاب کردن kernel32هرآنچه در سیستم فراخوانی می شود را چک کند.

برای دسترسی به توابع kernel32 راه های متعددی هست. بدون استفاده از IAT آن فراخوانی حساس را صدا می زنم و محصول شما را دور می زنم

برای فراخوانی سیستمی مستقیما از ntdll استفاده می کنیم. (مستند رسمی نیست اما قابل استفاده است)

HIDS شرکت ما همه فراخوانی های تک تک برنامه ها را در حافظه چک میکند
(با قلاب سازی جدول IAT)

شرکت ما تصمیم گرفته است با قلاب کردن kernel32هر آنچه در سیستم فراخوانی می شود را چک کند.

قلاب را در آخرین کتابخانه سطح کاربری سیستم عامل یعنی ntdll قرار می دهیم و از آنجا به فراخوانی ها نظارت می کنیم

برای دسترسی به توابع kernel32 راه های متعددی هست. بدون استفاده از IAT آن فراخوانی حساس را صدا می زنم و محصول شما را دور می زنم

برای فراخوانی سیستمی مستقیما از ntdll استفاده می کنیم. (مستند رسمی نیست اما قابل استفاده است)

```
mov r10, rcx
mov eax, ["SYSCALL_NUMBER_TO_INSERT"]
// here would be where a hook would be inserted, that jumps to code that belongs to the AV/EDR
syscall
ret
```

- ❑ این کد چه می کند؟
- ❑ این شماره از کجا یافت می شود؟
- ❑ ASLR چه تاثیری بر آن دارد؟

قلاب را در آخرین کتابخانه سطح کاربری سیستم عامل یعنی **ntdll** قرار می دهیم و از آنجا به فراخوانی ها نظارت می کنیم

حمله کننده نیازی به **ntdll** ندارد ، اگر خودش مستقیم فراخوانی را به کرنل انجام دهد.

شرکت ما تصمیم گرفته است
نظارت را از سطح هسته انجام دهد
تا به نتایج قابل اعتمادتری برسد.

انواع قلاب های سطح هسته برای میانگیری فراخوانی های سیستمی

Technique	OS examples	What it does	Privilege needed	Stability / compatibility	Detectability	Performance	Typical use
SSDT / syscall table hooking	Windows SSDT	Replace syscall table entries to intercept syscalls	Kernel driver, high	Very low on modern Windows (PatchGuard); breaks on updates	Easily detectable by tampering checks	Low overhead	Legacy rootkits / research
Inline kernel patching (code patching / inline hooks)	Windows/Linux	Overwrite function prologue to jump to handler	Kernel driver / module	Very fragile; breaks with patches, KASLR; can crash	Detectable via code integrity checks	Low latency	Rootkits, evasive hooks
Filter drivers (minifilter)	Windows minifilter	Register with I/O stack to observe/modify FS ops	Signed kernel driver with proper API	High — supported by OS	Designed to be visible/legitimate; harder to flag as malicious	Moderate, well-optimized	AV/IDS file and I/O monitoring
Kernel callbacks / notification APIs	Windows PsSetCreateProcessNotify, Linux security/LSM hooks	Use OS-provided callback APIs	Kernel driver / LSM module	Stable and forward-compatible	Expected/legitimate — low false-positive detect	Low overhead	Legitimate security products
Kernel modules hooking syscall table (Linux)	Linux syscall_table	Replace syscall table pointer entries	Kernel module, root	Fragile; many distros restrict module loading	Detectable; module listings	Low overhead	Research / rootkits (legacy)
kprobes / tracepoints / ftrace / eBPF	Linux kprobes, eBPF	Instrument kernel functions/tracepoints	Depends (eBPF needs kernel support, BPF syscalls)	Very stable if available	Less intrusive; can be audited but stealthy	Low to moderate; eBPF efficient	Production tracing/IDS (Falco, BPF-based monitors)

شرکت ما تصمیم گرفته است
نظارت را از سطح هسته انجام دهد
تابه نتایج قابل اعتمادتری برسد.

```
// Assume SSDT[0x25] is hooked
// But we find the real NtCreateFile in ntoskrnl
PVOID realNtCreateFile = FindOriginalFunction("NtCreateFile");

// Call it directly — bypasses SSDT entirely
status = ((NTSTATUS(*)(...))realNtCreateFile)(...);
```

اگرحمله کننده با نصب
یک درایور به سطح
کرنل وارد شد می تواند
برخی از این روش ها را
دور بزند.

جنگ بر سر قلاب ها

راه های عمومی دور زدن قلاب ها:

- ✓ -فراخوانی مستقیم و مستقل از تابع قلاب شده (که دیدیم)
- ✓ نگاشت مستقیم یک نسخه قلاب نشده از دیسک در حافظه برنامه بجای بار شدن پویا
- ✓ - قلاب گشایی (unhooking) و بازنویسی فراخوانی ها در حافظه با برداشتن قلاب ها

- تزریق مجدد هوک: تشخیص باز شدن هوک و اعمال مجدد هوک‌ها.
- ردیابی فراخوانی سیستمی: نظارت بر الگوهای غیرمعمول دستورالعمل فراخوانی سیستمی.
- یکپارچگی حافظه: اسکن DLLهای سیستمی اصلاح‌شده.
- فراخوان‌های هسته: استفاده از فراخوان‌های مجاز مایکروسافت (نه هوک‌ها) که به راحتی قابل دور زدن نیستند.
- نظارت مبتنی بر امکانات بیرون از سیستم عامل

ورود PatchGuard (KPP) به توهم پایان دوران هوک در کرنل

چرا توهم؟

پایان برای دفاع یا حمله؟!

در ویندوزهای ۳۲ بیتی، شرکت‌های آنتی‌ویروس دقیقاً مانند بدافزارها و روت‌کیت‌ها در کرنل هوک می‌گذاشتند:

- جدول فراخوانی سیستمی یعنی **SSDT** را هوک می‌کردند.
- توابع کرنل را Inline Hook می‌کردند تا هر فراخوانی را قبل از اجرا بررسی یا مسدود کنند.

اما این کار دو ایراد اساسی داشت:

1.صفحه آبی مرگ (BSOD) و بی‌ثباتی: اگر دو برنامه امنیتی هم‌زمان یک تابع کرنل را هوک می‌کردند، سیستم کرش می‌کرد.

2.برابری دسترسی با روت‌کیت‌ها: هیچ مرزی بین «هوک امنیتی» و «هوک مخرب» وجود نداشت.

از زمان ویندوز ۶۴ بیتی (۶۴x)، مایکروسافت مکانیزمی به نام **KPP (Kernel Patch Protection)** را معرفی کرد.

- کرنل ویندوز مرتباً جداول بحرانی خود (مانند SSDT، IDT، GDT) و بخش کدهای حساس سیستم‌عامل در حافظه کرنل را اسکن و هش می‌کند. اگر متوجه شود حتی یک بایت از کدهای هسته دستکاری یا هوک شده است، بلافاصله سیستم را با ارور **CRITICAL_STRUCTURE_CORRUPTION (BSOD)** ریستارت می‌کند تا از نفوذ جلوگیری شود.
- **قانون مایکروسافت برای EDRها:** مایکروسافت به هیچ شرکتی (حتی CrowdStrike یا Defender) استثنا نداده است. هیچ درایور امضاشده‌ای اجازه ندارد PatchGuard را دور بزند یا در کرنل هوک قرار دهد؛ اگر درایوری این کار را بکند، مایکروسافت امضای دیجیتال آن را باطل می‌کند.

مشکلات قلاب سنتی

- دور زدن اگر قلاب در سطح کاربری باشد، یک بدافزار ماهر با استفاده از فراخوانی مستقیم سیستم (Direct Syscall) یا برداشتن قلاب (Unhooking)آن را به راحتی دور می‌زند.
- حملات پایداری : اگر EDRبخواهد در لایه هسته قلاب بزند، یک باگ کوچک در کد EDR باعث Kernel Panicو سقوط کل سیستم سرور می‌شد.
- سربار پردازشی بالا

فراخوانی برگشتی (callback)

- مایکروسافت برای اینکه EDRها نابینا نشوند، توابع رسمی و استانداردی تحت عنوان Kernel Callbacks درون کرنل قرار داد.
- EDRها درایورهای خود را از طریق این APIهای قانونی ثبت می‌کنند تا سیستم‌عامل در زمان وقوع رویدادها، درایور EDRرا خبر کند
- ثبت فراخوانی برگشتی برای ایجاد پردازه، ایجاد نخ، بار شدنِ ماژول، درخواست دستگیره(Handle) به یک پردازه (مثلاً برای خواندن حافظه LSASSو): ObRegisterCallbacks
....

```
// شبه‌کد داخلی کرنل ویندوز
void PspInsertProcess(...) {
    // ... ساخت پروسس ...

    // حلقه روی آرایه کال‌بک‌ها
    for (int i = 0; i < 64; i++) {
        EX_FAST_REF FastRef = PspCreateProcessNotifyRoutine[i];
        if (FastRef != NULL) {
            // ماسک کردن ۴ بیت آخر برای رسیدن به آدرس واقعی
            PEX_CALLBACK_ROUTINE_BLOCK Block = ExGetObjectFastRef(FastRef);

            // فراخوانی مستقیم تابع EDR!
            Block->CallbackRoutine(Process, ProcessId, CreateInfo);
        }
    }

    // ... ادامه روال اجرای پروسس
}
```

تمرین کلاسی

- برای هریک از مسیرهای دورزدن رصد فراخوانی سیستم که گفته شد در ویندوز، یک بدافزار مثال بزنید که چنین کرده است.
- مشابه همین مسیری که برای رصد فراخوانی های سیستمی برای ویندوز رفتیم برای یک EDR اندروید بررسی کنید (سطح کاربری و سطح هسته)
  - یک شمای کلی از معماری اندروید
  - مسیر رپرها در یک فراخوانی سیستم
  - امکانات قابل استفاده برای EDR
  - عمیقترین نقطه قابل استقرار برای EDR
  - مقایسه با ویندوز
  - تحلیل مطالب فوق
- مهلت یک هفته

مساله چه بود؟(در لینوکس)

- من می خواهم یک روتین در جایی در کرنل قلاب کنم
- اولا هرجایی ممکن نیست، چون من هرجابرچسب export_symbol را دارد می توانم قلاب بیندازم و بقیه نقاط کرنل اساسا اجازه قلاب نمی دهد
- در به روزرسانی ها همین ساختاری که قلاب کردی ممکن است تغییر کند و سیستم با این برنامه پایین بیاید.
- فرض کن می‌خواستی یک فایروال یا سیستم لودبالانسر فوق‌سریع بنویسی:
- تا قبل از آمدن قابلیت **XDP**، درایورهای کارت شبکه هیچ نقطه‌ای نداشتند که به تو بگویند: «بفرما، این بایت‌های خام بسته، قبل از اینکه به صف لینوکس برود، در موردش تصمیم بگیر».
- کرنل ابتدا خودش پکت را در صف‌های پیچیده می‌گذاشت، کلی حافظه تخصیص می‌داد، و تازه در لایه Netfilter به تو می‌گفت حالا دراختیار تو اگر قلابی لازم داری.
- تو به عنوان نویسنده درایور، **مجبور بودی خود لینوکس را قانع کنی** که معماری شبکه را تغییر دهد و قلاب جدیدی به نام XDP در درایورهای اینتل و Mellanox تعبیه کند تا امکان مداخله فوق‌سریع به وجود بیاید .

انقلاب eBPF

«کرنل لینوکس تا پیش از eBPF، به دلیل حفظ پایداری و امنیت، تغییرناپذیرترین بخش سیستم بود. eBPFاین پارادوکس تاریخی بین "پایداری سیستم‌عامل" و "سرعت نوآوری نرم‌افزار" را برای همیشه حل کرد. هسته اکنون می‌تواند به هر نیاز امنیتی یا زیرساختی بدون دست زدن به کدهای پایه‌ای خود پاسخ دهد.
مبدعان ebpf

فناوری eBPFرویکرد صنعت را به سیستم‌عامل دگرگون کرده است. این تکنولوژی هسته را از یک جعبه سیاه بسته و صلب، به یک بستر باز، سریع و ایمن برای توسعه نرم‌افزارهای مانیتورینگ، شبکه و امنیت بدل کرده است.

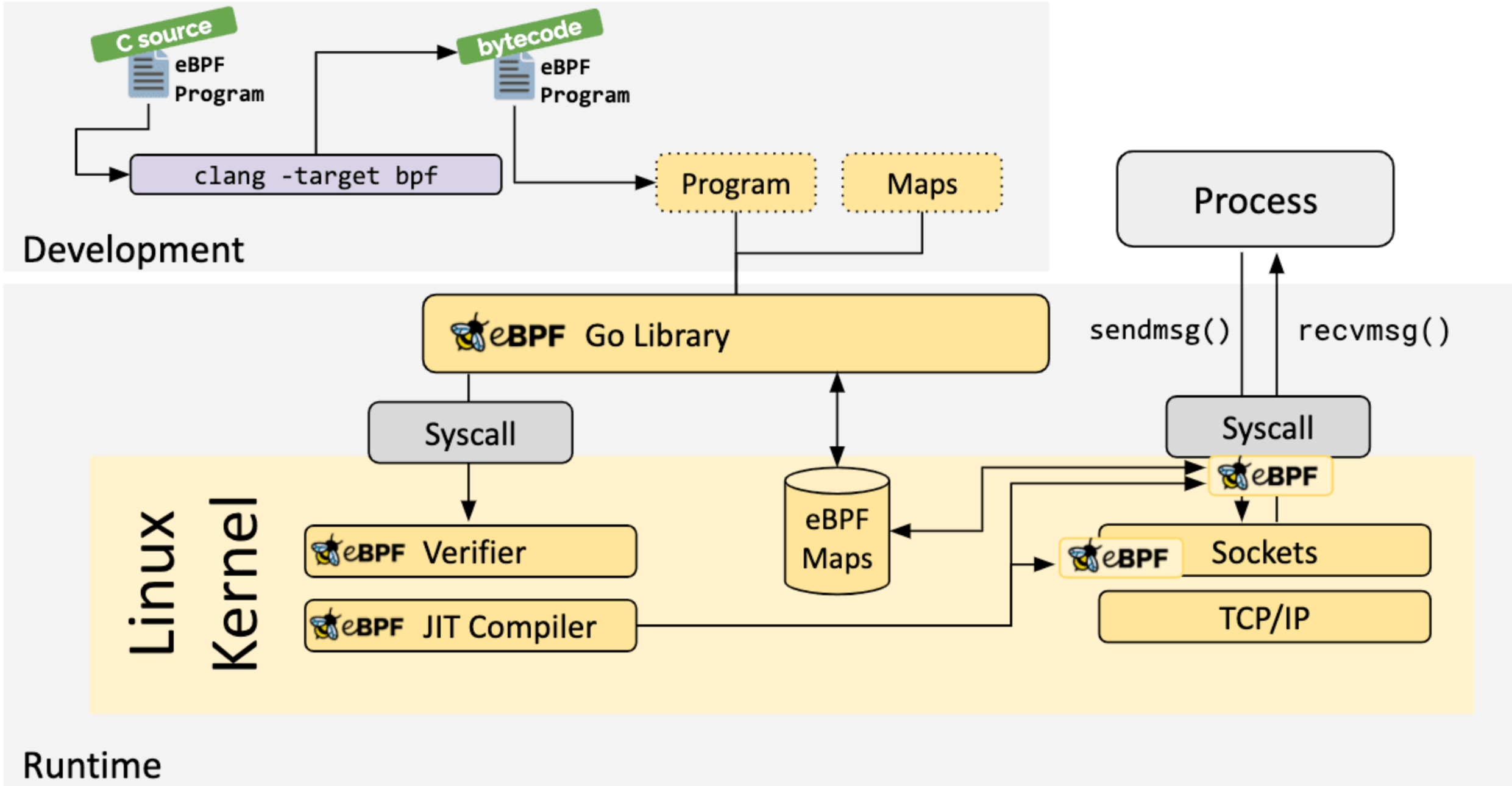
مارک روسینوویچ

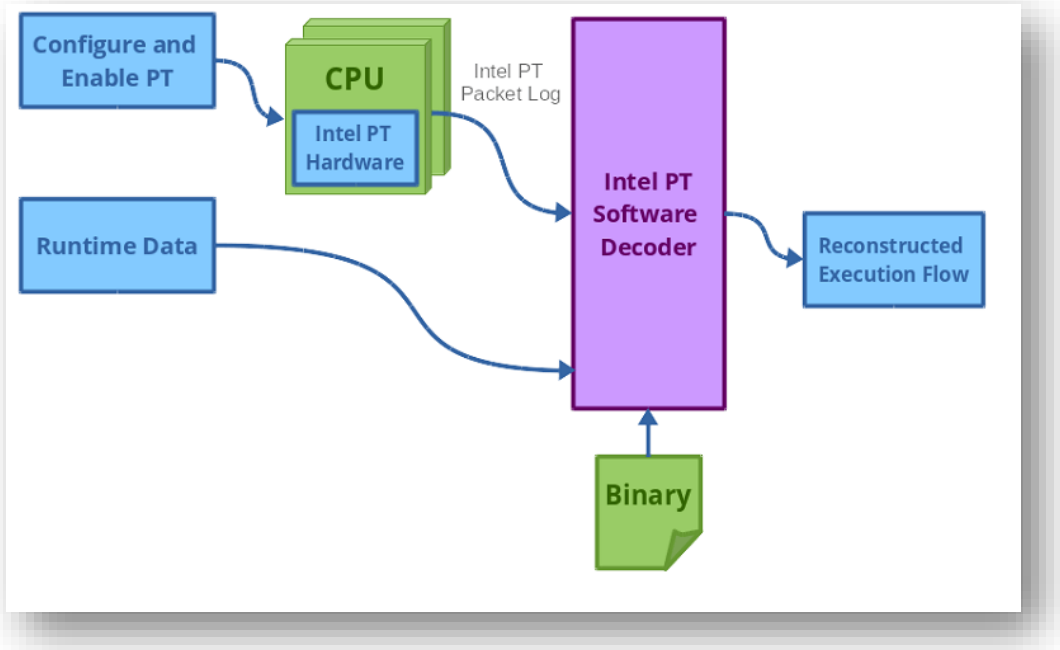
فناوری BPF/eBPFبزرگ‌ترین تغییری است که در طی دهه‌ها بر روی هسته لینوکس اعمال شده است. پیش از این، برای به دست آوردن رصدپذیری عمیق یا باید ماژول‌های پرریسک می‌نوشتید یا منتظر می‌ماندید تا تغییرات شما سال‌ها بعد در نسخه بعدی کرنل پذیرفته شود. eBPFبه ما نوعی ابرقدرت (Superpower) داده است: توانایی دیدن و تغییر هر بسته داده، هر سیستم‌کال و هر تابع سیستمی، آن هم بدون کوچک‌ترین افت کارایی.
برندن گرگ از برجسته‌ترین معمار پرفورمنس کامپیوتر

همان‌طور که جاوااسکریپت صفحات وب را از اسناد متنی ایستا (Static HTML) به برنامه‌های تعاملی و پویا تبدیل کرد، **eBPF نیز هسته سیستم‌عامل لینوکس را برنامه‌پذیر کرد.** بدون نیاز به بازکامپایل کردن کرنل یا بارگذاری ماژول‌های خطرناک، eBPF به شما اجازه می‌دهد رفتار کرنل را در زمان واقعی بازتعریف کنید؛ این دقیقاً همان نقطه عطفی است که وب سال‌ها پیش تجربه کرد.
لیز رایس

eBPF دقیقا چطور کار می کند؟

- ایده اصلی:لینوکس یک زبان واسط (ISA)ساده و امن درون خودش تعریف کرد تا بررسی کدهای اضافی (قلاب‌ها) راحت و بی‌خطر باشد.
- قرارداد لینوکس با توسعه‌دهنده:کدت را با این ISAکامپایل کن و بیاور بده به من. من زمان بارگذاری چکش می‌کنم؛ اگر درست و ایمن بود، می‌گذارمش سر جایش!
- به‌مرور زمان، راهکارهای اصولی برای زیاد کردن نقاط امنِ قابل‌قلاب‌کردن در بخش‌های مختلف کرنل فراهم شد.
- نقشِ JIT Compiler: بایت‌کدِ تاییدشده، در زمان بارگذاری توسط JITبه باینری واقعی سخت‌افزار (مثلاً اینتل یا ARM)تبدیل میَ‌شود.
- چرا از قبل تبدیلش نکنیم؟ چون معماری سخت‌افزار و مشخصات هر سرور با دیگری فرق می‌کند؛ پس تبدیلِ نهایی باید دقیقاً روی همان سیستم و هنگام ورود انجام شود. همچنین اطلاعات زمان اجرا به سیاستَ ها کمک می کند

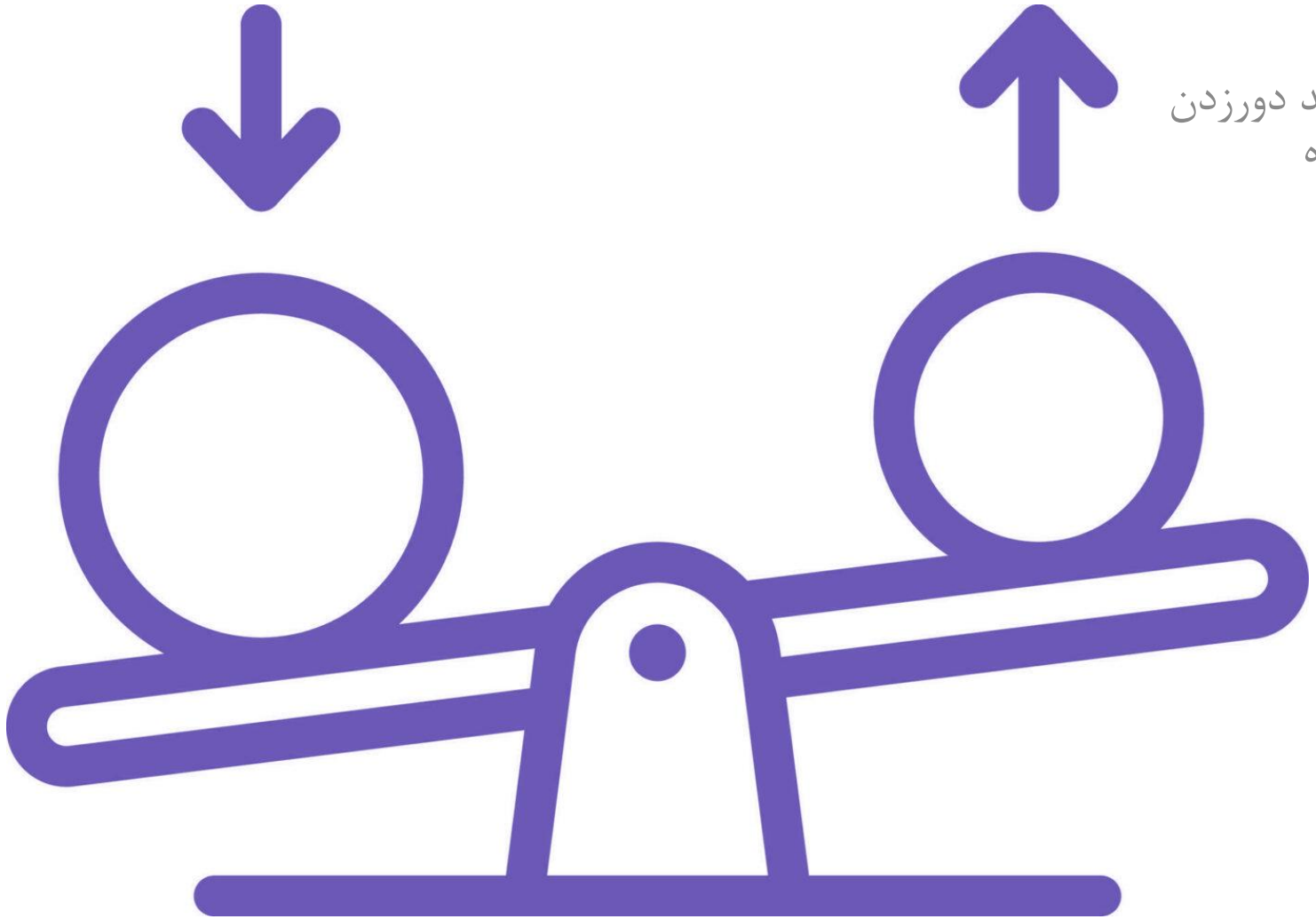

C source
eBPF
Program
bytecode
eBPF
Program
clang -target bpf
Program
Maps
Development
Process
eBPF Go Library
sendmsg()
recvmsg()
Syscall
Syscall
Linux
Kernel
eBPF
Maps
eBPF
eBPF Verifier
eBPF
Sockets
eBPF JIT Compiler
TCP/IP
Runtime



امکانات پردازنده برای میانگیری فراخوانی های سیستمی

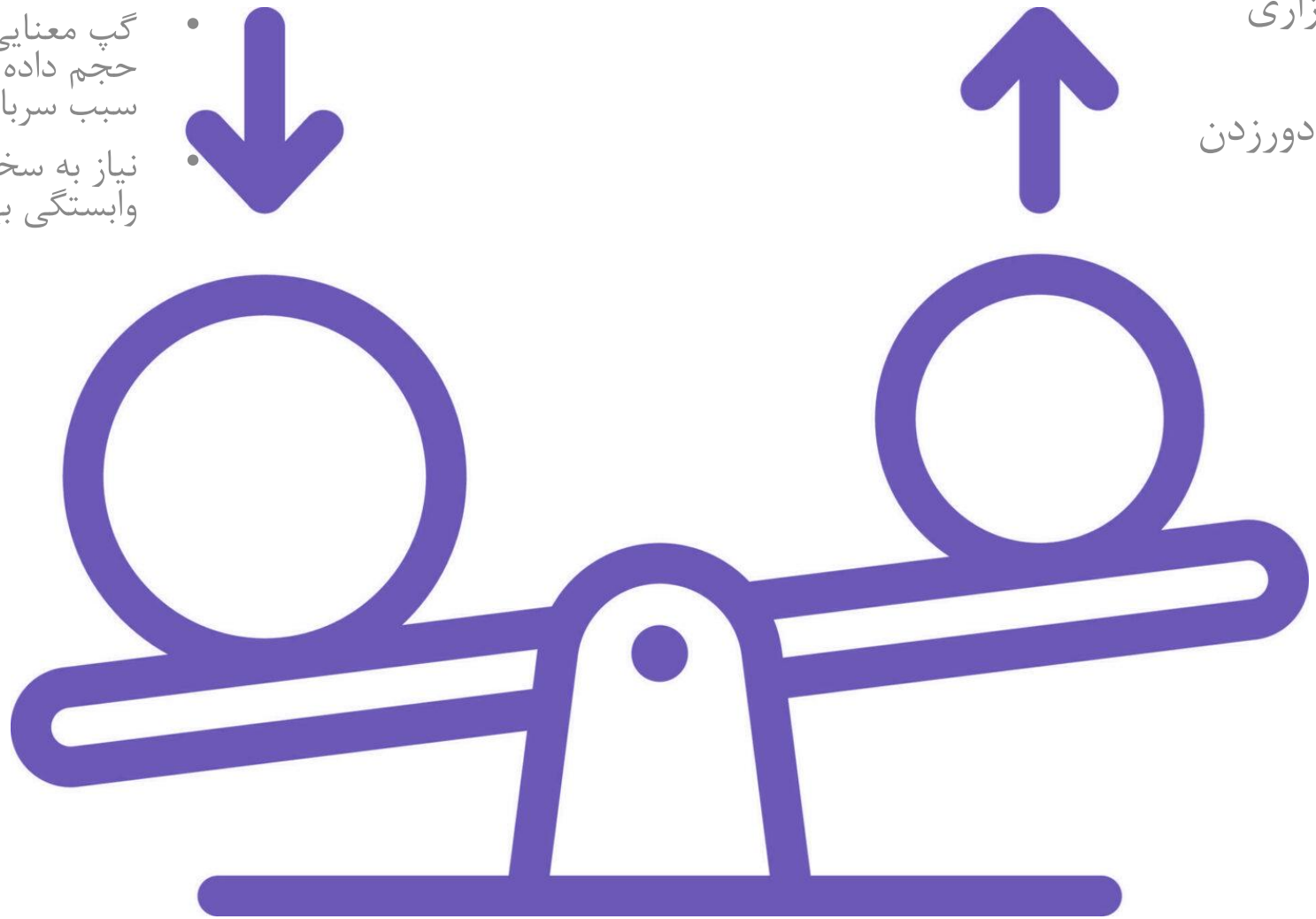
چرا از امکانات سخت افزاری در تشخیص نفوذ کمتر بهره برداری می شود؟

- پیاده سازی سخت افزاری وسریع
- پیچیده بودن فرآیند دورزدن توسط حمله کننده



چرا از امکانات سخت افزاری در تشخیص نفوذ کمتر بهره برداری می شود؟

- پیاده سازی سخت افزاری وسریع
- پیچیده بودن فرآیند دورزدن توسط حمله کننده



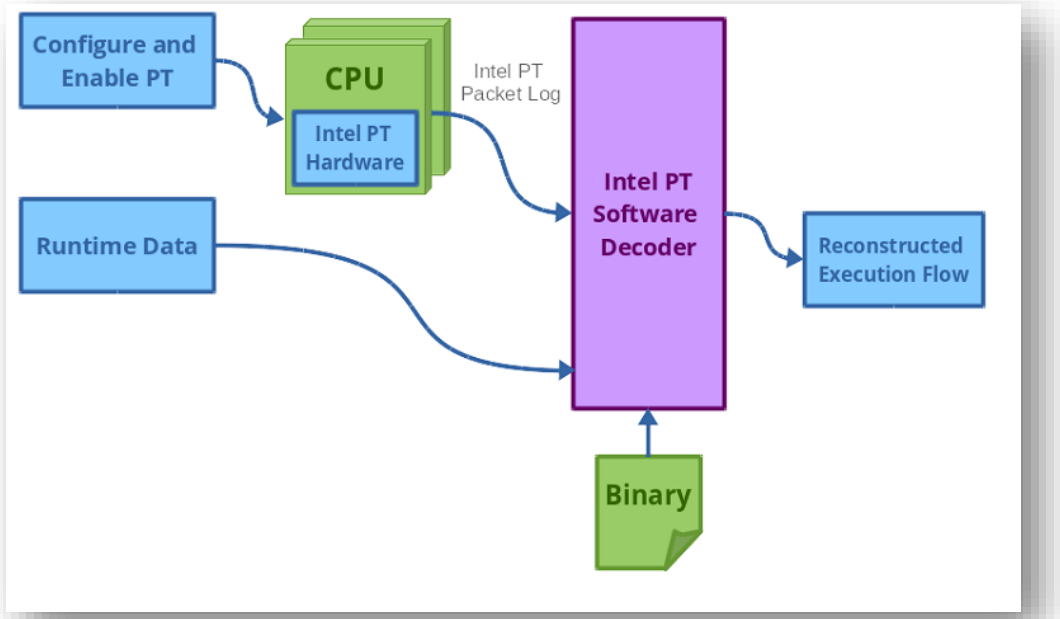
- گپ معنایی در عین بالا بودن حجم داده ها درسطح سخت افزار سبب سرباز پردازشی بالا می شود
- نیاز به سخت افزارهای جدیدتر و وابستگی بهپردازنده خاص

شکاف معنایی

شکاف معنایی به عدم ارتباط بین داده‌های سطح پایین جمع‌آوری‌شده توسط سخت‌افزار (مانند Intel PT) و معنا یا هدف سطح بالایی که یک سیستم امنیتی می‌خواهد بفهمد - مانند «آیا این یک فراخوانی سیستمی مخرب بود؟» - اشاره دارد.

Intel PT رویدادهای جریان کنترل (شاخه‌ها، پرش‌ها، فراخوانی‌ها) را ثبت می‌کند، اما مستقیماً به شما نمی‌گوید:

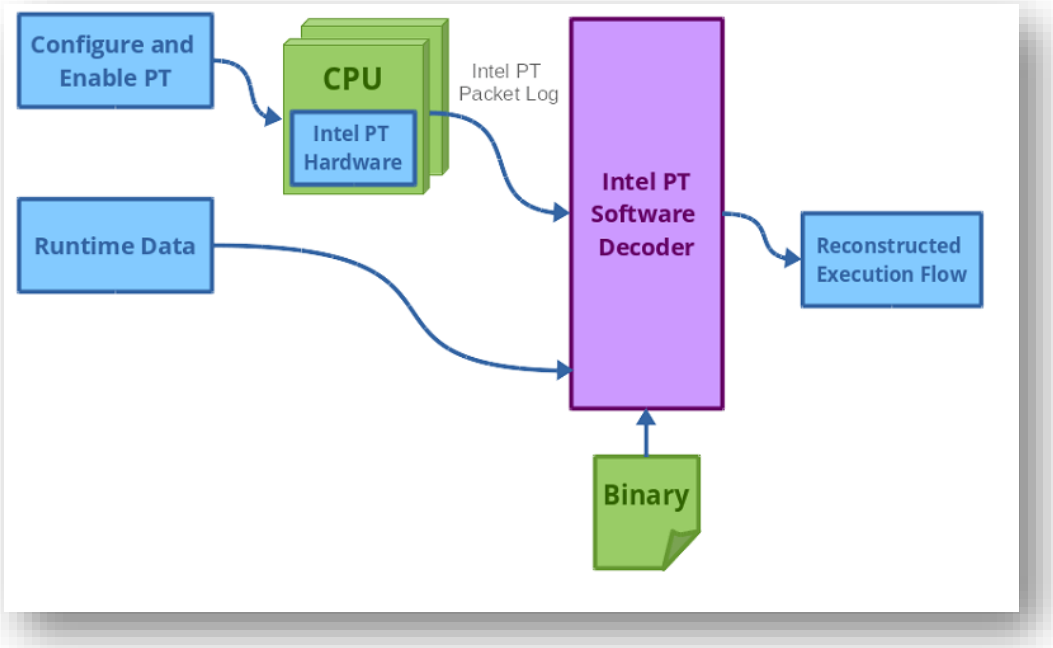
- ✓ کدام فراخوانی سیستمی فراخوانی شده است
- ✓ چه آرگومان‌هایی ارسال شده است
- ✓ چه چیزی از فراخوانی سیستمی برگردانده شده است
- ✓ آیا مخرب بوده است یا خوش‌خیم



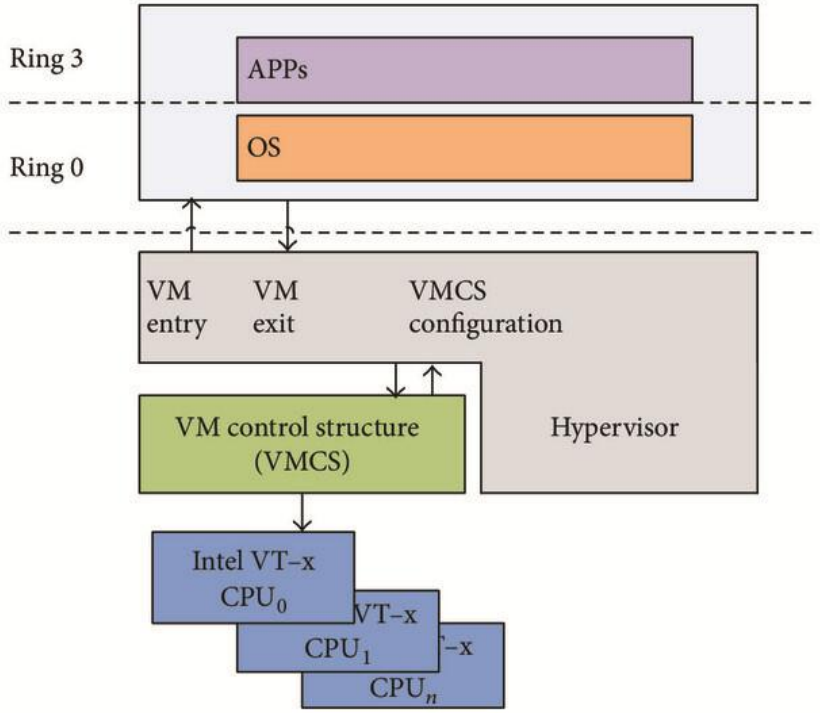
امکانات پردازنده برای میانگیری فراخوانی های سیستمی

میشه دورش زد؟

Evasion Method	Description	Difficulty
Disable PT	Modify MSRs or kernel config	High
Execute outside scope	Use untraced regions or threads	Medium
Self-modifying code	Obscure control flow	Medium
Buffer flooding	Overwhelm trace output	Medium
Timing-based evasion	Detect PT presence via latency	Low–Medium

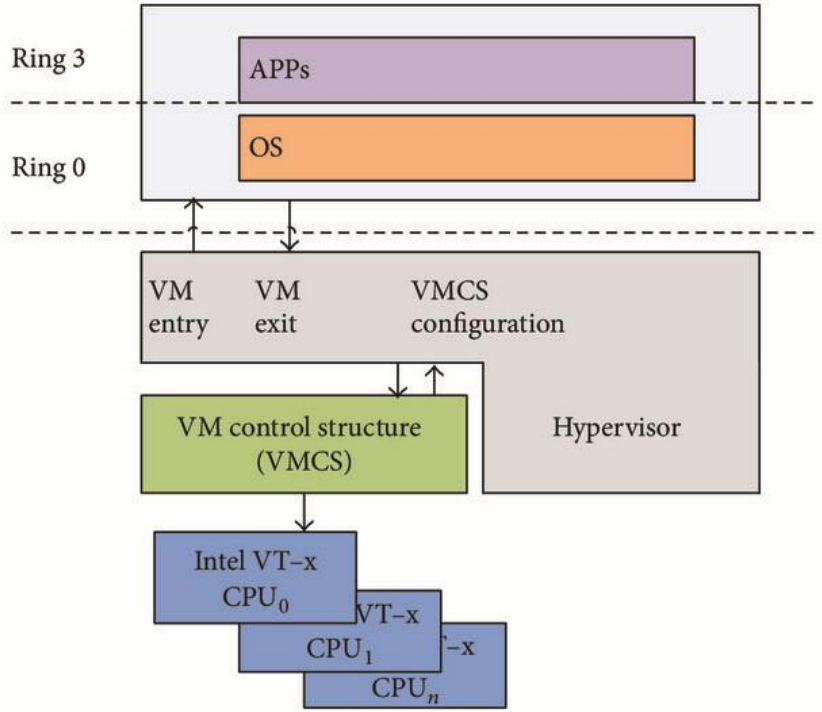


امکانات پردازنده برای میانگیری فراخوانی های سیستمی



- ✓ VMCS (ساختار کنترل ماشین مجازی) وضعیت CPUماشین مجازی و کنترل‌هایی را که تعیین می‌کنند کدام رویدادها باعث خروج ماشین مجازی می‌شوند، در خود نگه می‌دارد.
- ✓ ناظر، VMCSرا طوری پیکربندی می‌کند که مواردی مانند دستورالعمل‌های syscall، بارهای CR3یا دسترسی‌های MSRرا به دام بیندازد.
- ✓ هنگامی که یکی از این رویدادها رخ می‌دهد، CPUیک VMEXITانجام می‌دهد و وضعیت ماشین مجازی را در VMCSذخیره می‌کند.

درون بینی ماشین های مجازی برای میانگیری فراخوانی های سیستمی



- ✓ VMCS (ساختار کنترل ماشین مجازی) وضعیت CPUماشین مجازی و کنترل‌هایی را که تعیین می‌کنند کدام رویدادها باعث خروج ماشین مجازی می‌شوند، در خود نگه می‌دارد.
- ✓ ناظر، VMCSرا طوری پیکربندی می‌کند که مواردی مانند دستورالعمل‌های syscall، بارهای CR3یا دسترسی‌های MSRرا به دام بیندازد.
- ✓ هنگامی که یکی از این رویدادها رخ می‌دهد، CPUیک VMEXITانجام می‌دهد و وضعیت ماشین مجازی را در VMCSذخیره می‌کند.

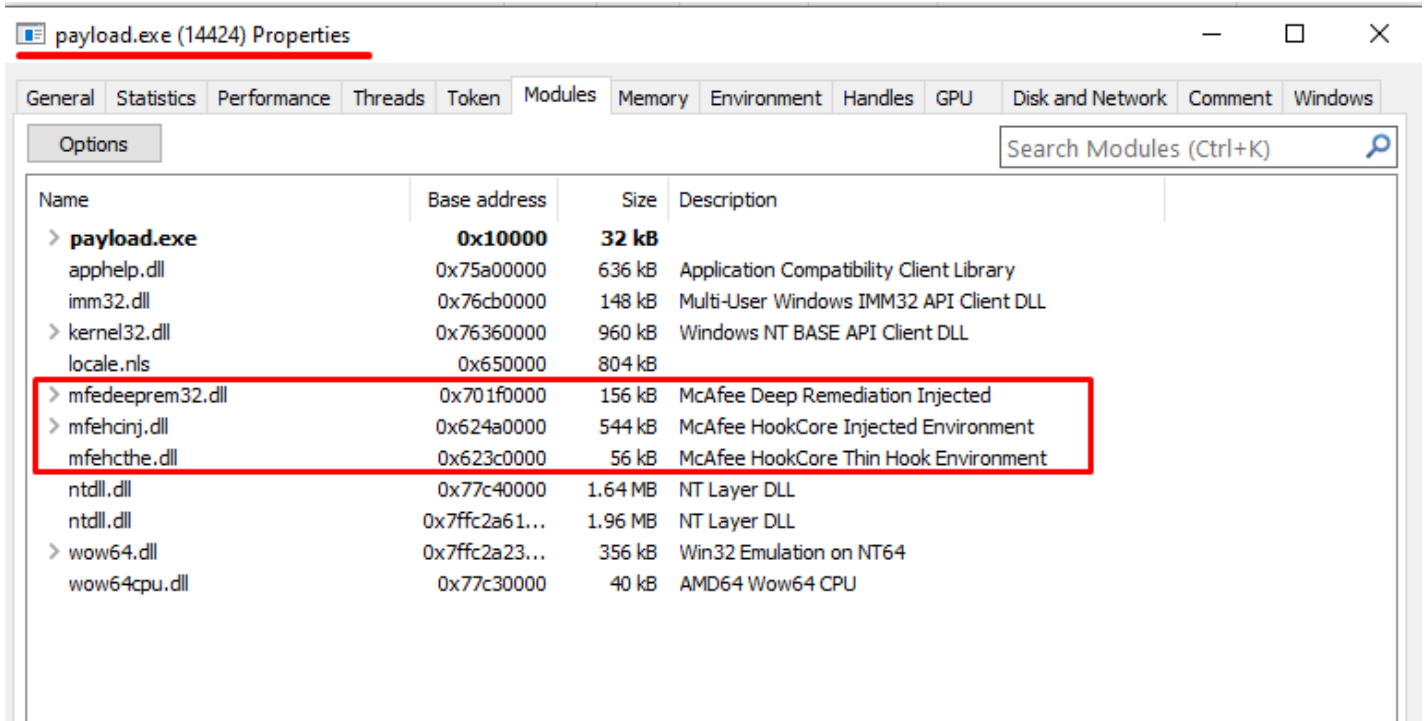
میشه دورش زد؟

درون بینی ماشین های مجازی برای میانگیری فراخوانی های سیستمی

راه های متنوعی هست و هم توسط حمله کنندگان محدودی و هم توسط مقالات دور زده شده ساده ترین ایده گم کردن فراخوانی ها بین فراخوانی هایی هست که خود نخ های سیستم عامل درون ماشین مجازی انجام می دهند.

چطور به حافظه برنامه دسترسی دارد؟

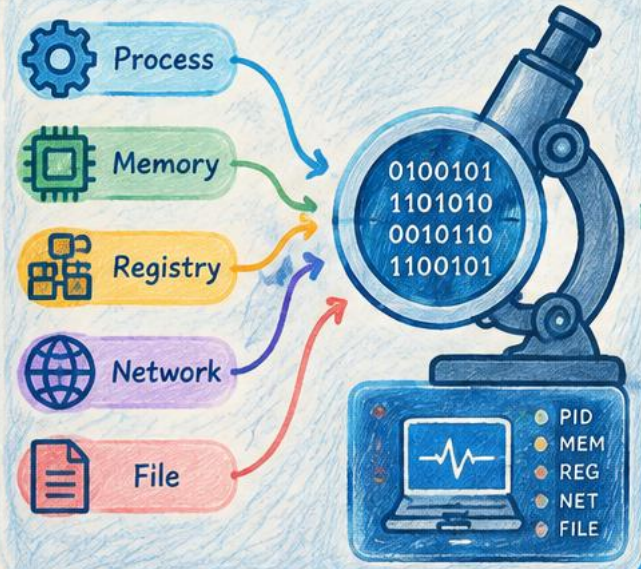
- تزریق کتابخانه رصدگر به برنامه هدف
- هر برنامه ای که ایجاد می شود، ابتدا کتابخانه متعلق به سیستم تشخیص نفوذ یا هر نرم افزار امنیتی در آن تزریق می شود.
- فکر می کنید به چه طریق این کار انجام می شود ؟
- قلاب سازی به فرآیند ایجاد برنامه!
- دیگر روش ها؟



پایپ‌لاین تشخیص EDR

1 تله‌متری دقیق (دید میکروسکوپی)

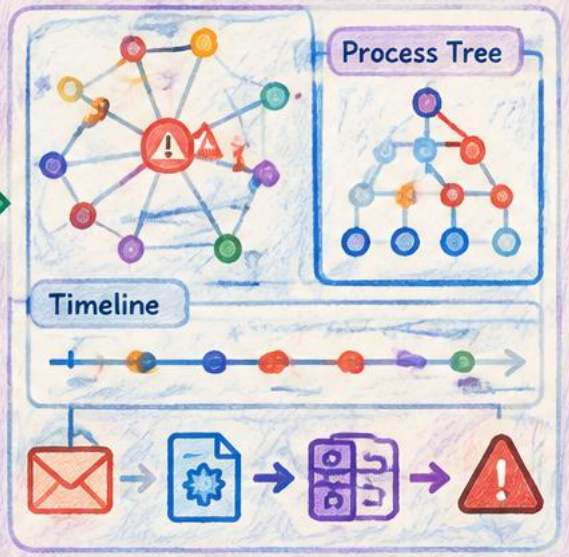
جمع‌آوری جزئیات ریز و رویدادهای اتمی از تفاط پایانی سیستم



2 نرمال‌سازی و پیش‌پردازش

3 همپستگی رخدادها (دید ماکروسکوبیک)

اتصال رویدادهای منام و ایجاد یک داستان منسجم از حمله



4 موتورهای تشخیص



- ماشین لرنینگ
- قوانین رفتاری
- تحلیل علائم
- هوش تهدید

5 پاسخ و اقدام

- مسدودسازی
- جداسازی سیستم
- حذف تهدید
- گزارش و مستندسازی



دید عمیق‌تر + تحلیل هوشمندانه = امنیت بیشتر



با وجود ناکارآمدی در تجمیع گرافی داده ها، صنعت واقعا چه می کند؟

- درحت های LSM تا ۲۰۲۲: https://www.crowdstrike.com/en-us/blog/how-log-structured-merge-trees-enable-crowdstrike-to-process-trillions-of-events-per-day/
- اوضاع ناکارآمد گراف ها در ۲۰۲۵: https://www.usenix.org/system/files/usenixsecurity25-bilot.pdf
- معماری جدیدی که فالکون خریده است: https://www.crowdstrike.com/en-us/blog/how-humio-leverages-kafka-and-brute-force-search-to-get-blazing-fast-search-results/